The Fundamentals of NLP

NLP

In the vast landscape of technological advancements, Natural Language Processing (NLP) serves as a crucial link, bridging the gap between human communication and machine understanding. Let’s delve into the essentials of NLP you need to know to start your project.

What is Natural Language Processing (NLP)

Natural language processing (NLP) is the discipline of building machines that can manipulate human language — or data that resembles human language — in the way that it is written, spoken, and organized.

It evolved from computational linguistics, which uses computer science to understand the principles of language, but rather than developing theoretical frameworks, NLP is an engineering discipline that seeks to build technology to accomplish useful tasks.

NLP can be divided into two overlapping subfields:

Natural language understanding (NLU), which focuses on semantic analysis or determining the intended meaning of text, and

Natural language generation (NLG), which focuses on text generation by a machine. NLP is separate from — but often used in conjunction with — speech recognition, which seeks to parse spoken language into words, turning sound into text and vice versa

What are the fundamental Concepts of NLP?

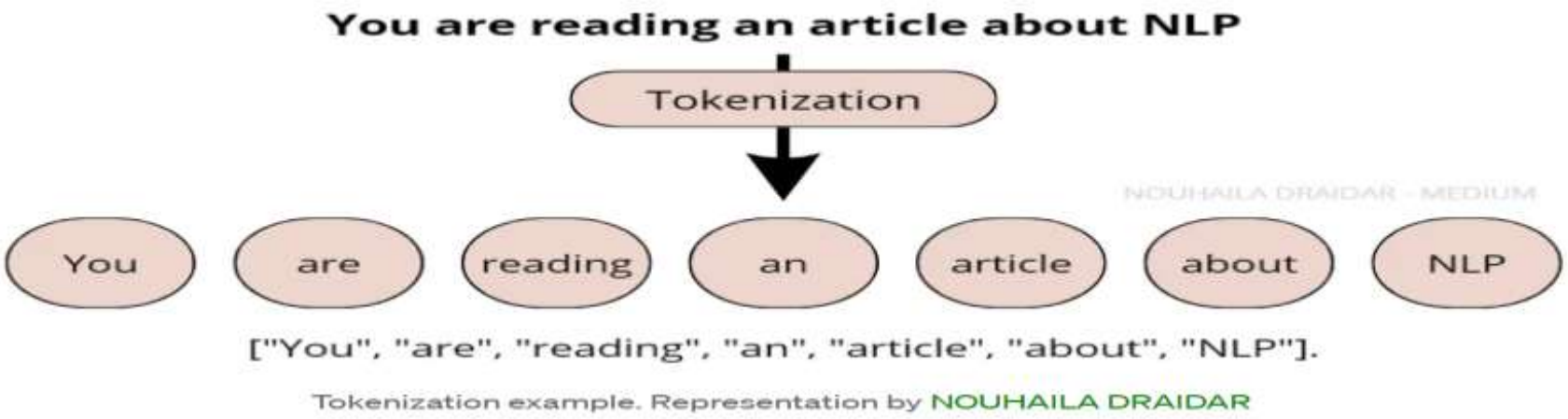
Here are some fundamental concepts before starting your NLP project.

Preprocessing

Like any other data science project, preprocessing is fundamental in NLP. it can involve removing punctuation, stop words, tokenization, Part-of-Speech Tagging, lemmatization, stemming, and much more.

Tokenization

It is the act of breaking down a text into individual units, usually words or phrases, these fragments named tokens, enable machines to navigate and understand the complexities of human language.



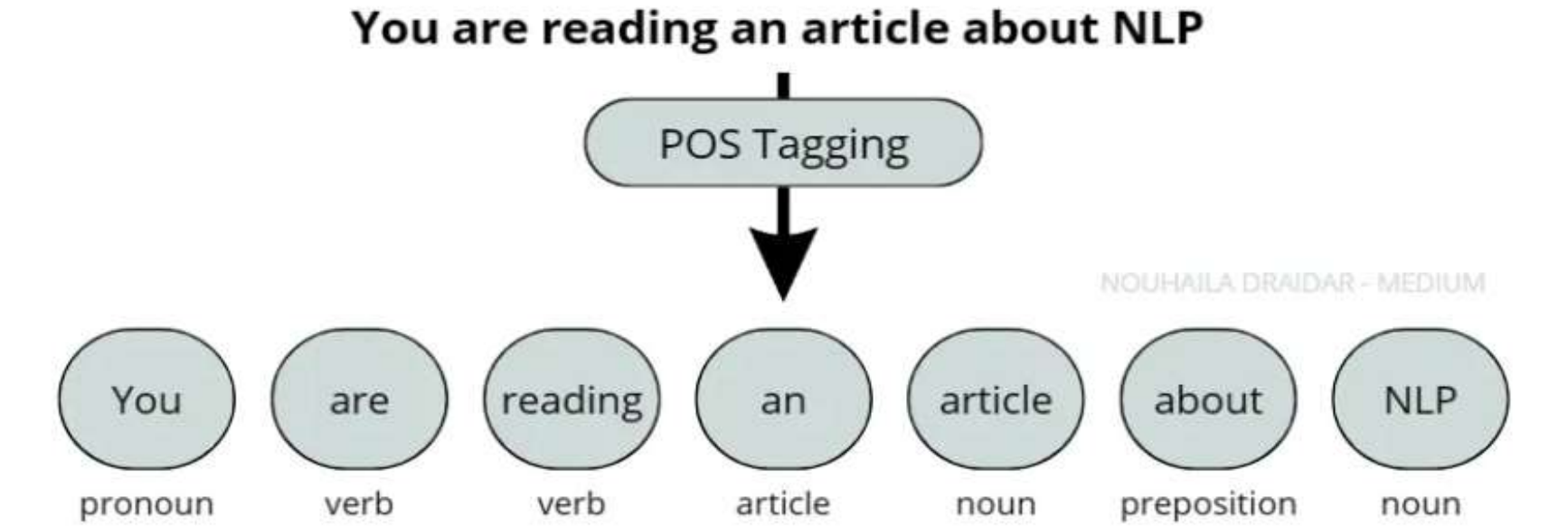
Here are some fundamental concepts before starting your NLP project.

Preprocessing

Like any other data science project, preprocessing is fundamental in NLP. it can involve removing punctuation, stop words, tokenization, Part-of-Speech Tagging, lemmatization, stemming, and much more.

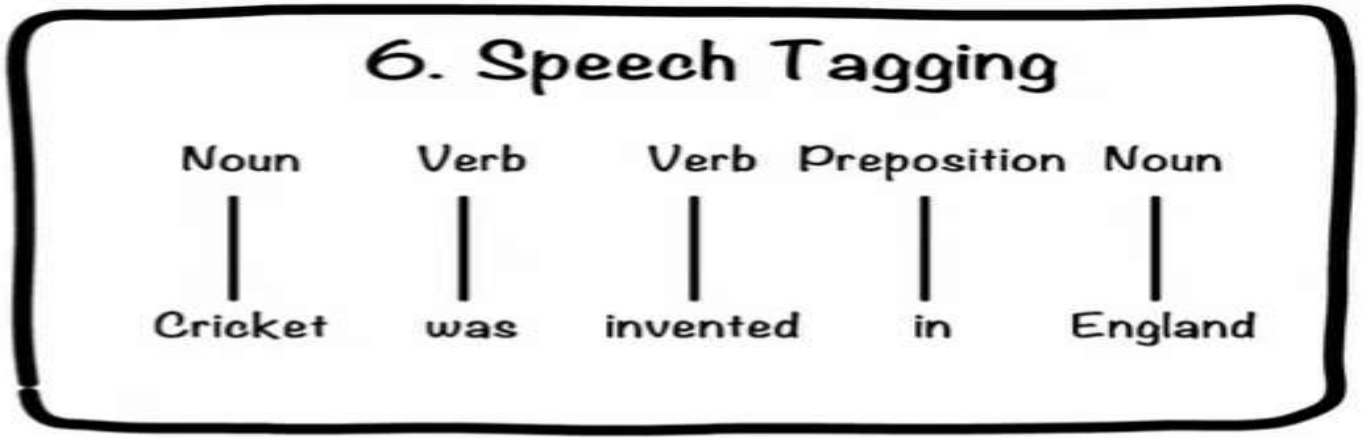
Tokenization

It is the act of breaking down a text into individual units, usually words or phrases, these fragments named tokens, enable machines to navigate and understand the complexities of human language.



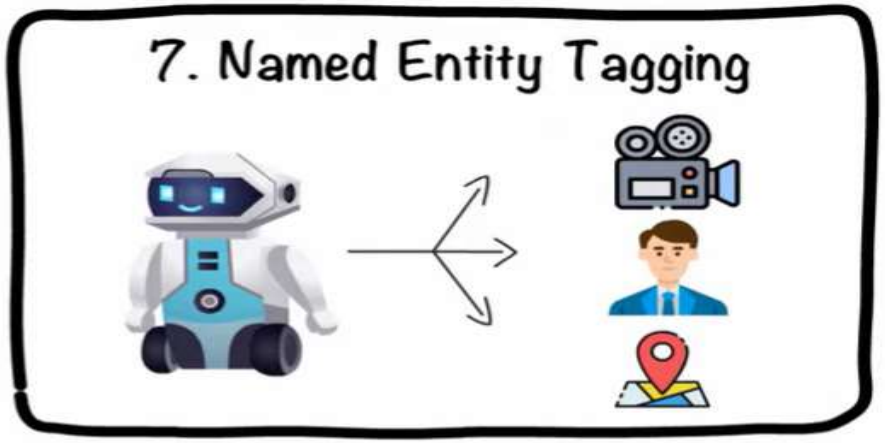
["You" (pronoun), "are" (verb), "reading" (verb), "an" (article), "article" (noun), "about" (preposition), "NLP" (noun)]

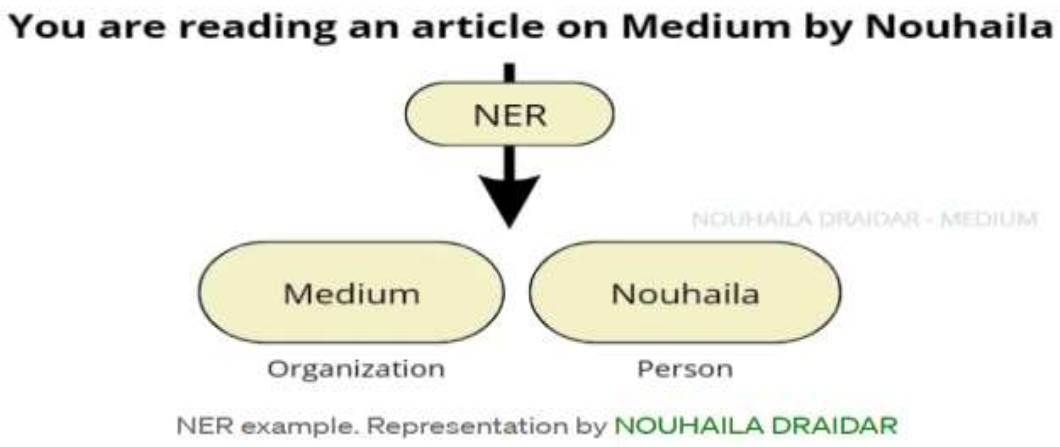
POS Tagging example. Representation by NOUHAILA DRAIDAR



Named Entity Recognition (NER)

Imagine reading a story where every character, place, and organization is highlighted. NER does exactly that, categorizing entities such as names, locations, and organizations, which is how machines can unravel the story within the text.





How does natural language processing work?

Natural language processing (NLP) combines computational linguistics, machine learning, and deep learning models to process human language.

Computational linguistics

Computational linguistics is the science of understanding and constructing human language models with computers and software tools.

Machine learning

Machine learning is a technology that trains a computer with sample data to improve its efficiency. Human language has several features like sarcasm, metaphors, variations in sentence structure, plus grammar and usage exceptions that take humans years to learn. Programmers use machine learning methods to teach NLP applications to recognize and accurately understand these features from the start.

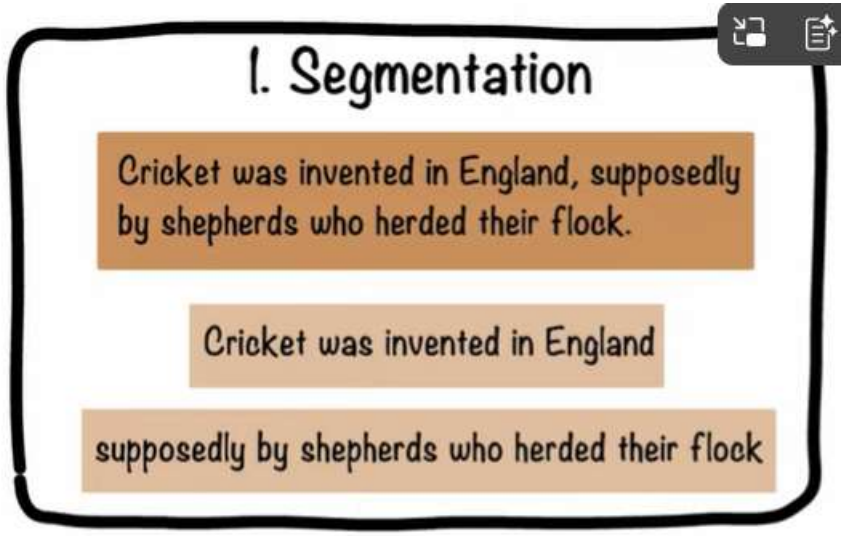
Deep learning

Deep learning is a specific field of machine learning which teaches computers to learn and think like humans. It involves a neural network that consists of data processing nodes structured to resemble the human brain. With deep learning, computers recognize, classify, and co-relate complex patterns in the input data.

NLP implementation steps

Typically, NLP implementation begins by gathering and preparing unstructured text or speech data from sources like cloud data warehouses, surveys, emails, or internal business process applications.

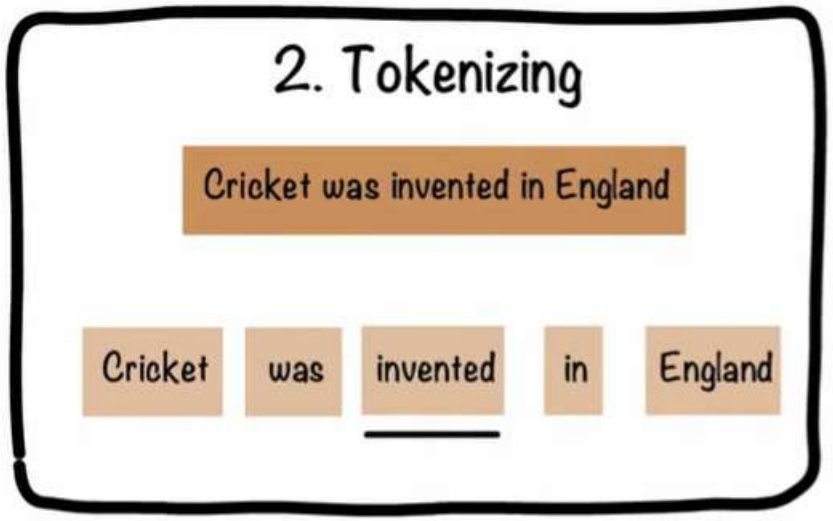
Pre-processing



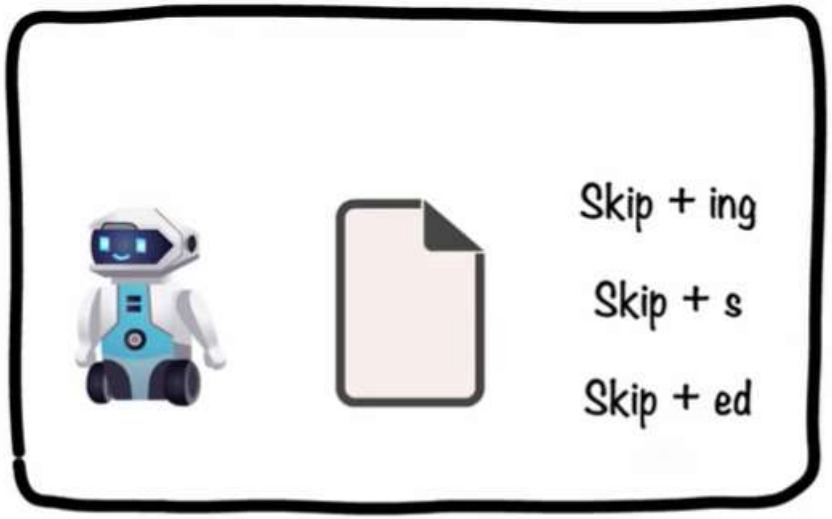
The NLP software uses pre-processing techniques such as tokenization, stemming, lemmatization, and stop word removal to prepare the data for various applications.

Here's a description of these techniques:

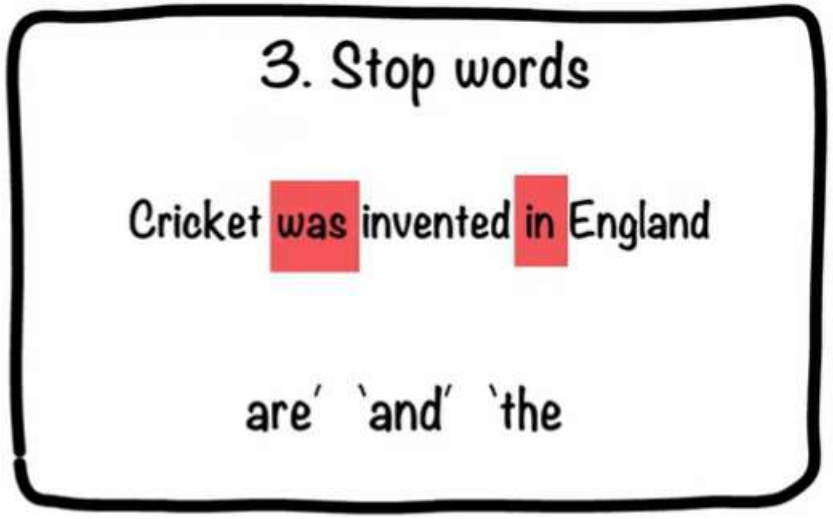
- Tokenization breaks a sentence into individual units of words or phrases.



- Stemming and lemmatization simplify words into their root form. For example, these processes turn "starting" into "start."



- Stop word removal ensures that words that do not add significant meaning to a sentence, such as "for" and "with," are removed.



Training

Researchers use the pre-processed data and machine learning to train NLP models to perform specific applications based on the provided textual information. Training NLP algorithms requires feeding the software with large data samples to increase the algorithms' accuracy.

Deployment and inference

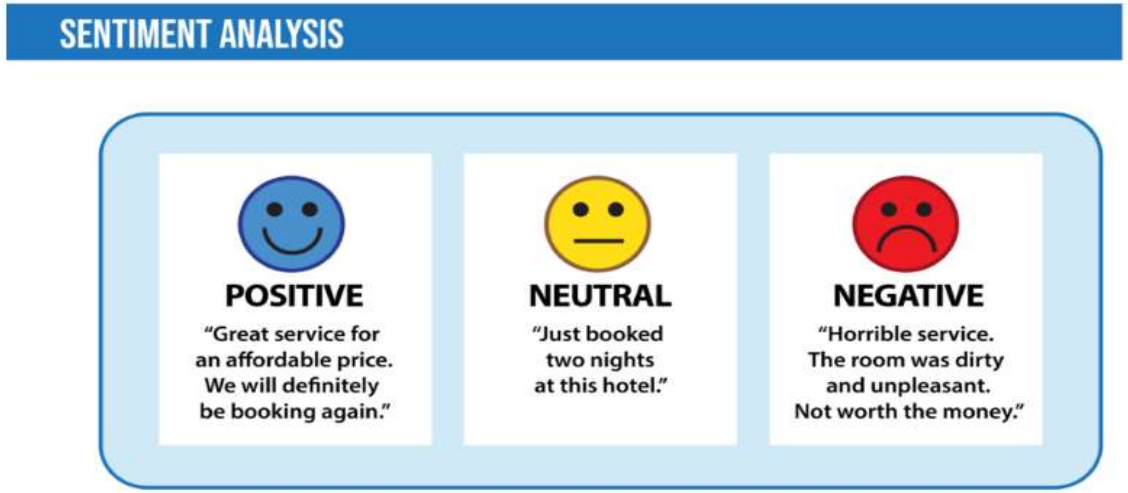
Machine learning experts then deploy the model or integrate it into an existing production environment. The NLP model receives input and predicts an output for the specific use case the model's designed for. You can run the NLP application on live data and obtain the required output.

NLP Tasks or NLP Applications

NLP is used for a wide variety of language-related tasks, including answering questions, classifying text in a variety of ways, and conversing with users.

Text Classification

Text classification is a supervised learning task where the goal is to assign predefined categories or labels to text based on its content using supervised learning algorithms, such as Support Vector Machines (SVM) or deep learning models.



Given text, sentiment analysis classifies its emotional quality.

- **Sentiment analysis** is the process of classifying the emotional intent of text. Generally, the input to a sentiment classification model is a piece of text, and the output is the probability that the sentiment expressed is positive, negative, or neutral.

- Typically, this probability is based on either hand-generated features, word n-grams, TF-IDF features, or using deep learning models to capture sequential long- and short-term dependencies.
- Sentiment analysis is used to classify customer reviews on various online platforms as well as for niche applications like identifying signs of mental illness in online comments.

NAMED ENTITY RECOGNITION (NER) TAGGING

- **Named entity recognition** aims to extract entities in a piece of text into predefined categories such as personal names, organizations, locations, and quantities.
- The input to such a model is generally text, and the output is the various named entities along with their start and end positions.
- Named entity recognition is useful in applications such as summarizing news articles and combating disinformation. For example, here is what a named entity recognition model could provide:

Summarization is the task of shortening text to highlight the most relevant information. Researchers at Salesforce developed a summarizer that also evaluates factual consistency to ensure that its output is accurate. Summarization is divided into two method classes:

- **Extractive summarization** focuses on extracting the most important sentences from a long text and combining these to form a summary. Typically, extractive summarization scores each sentence in an input text and then selects several sentences to form the summary.
- **Abstractive summarization** produces a summary by paraphrasing. This is similar to writing the abstract that includes words and sentences that are not present in the original text. Abstractive summarization is usually modeled as a sequence-to-sequence task, where the input is a long-form text and the output is a summary.

Question answering deals with answering questions posed by humans in a natural language. One of the most notable examples of question answering was Watson, which in 2011 played the television game-show *Jeopardy* against human champions and won by substantial margins. Generally, question-answering tasks come in two flavors:

- **Multiple choice:** The multiple-choice question problem is composed of a question and a set of possible answers. The learning task is to pick the correct answer.
- **Open domain**: In open-domain question answering, the model provides answers to questions in natural language without any options provided, often by querying a large number of texts.

- **Machine translation** automates translation between different languages. The input to such a model is text in a specified source language, and the output is the text in a specified target language.

Why Is GenAI Important?

GenAI is important because it is transforming a wide variety of industries by automating content creation, enhancing personalization, and even driving innovation. Key benefits include:

Automated Content Generation: Efficiently produces increasingly high-quality text, images, and code.

Enhanced Creativity: Can assist in such diverse capacities as design, art, music, and storytelling.

Personalized User Experiences: Tailors content for individuals based on preferences.

Improved Efficiency: Reduces human effort in repetitive tasks like data augmentation and summarization.

Accelerated Innovation: Can rapidly and effectively assist in the development of new applications.

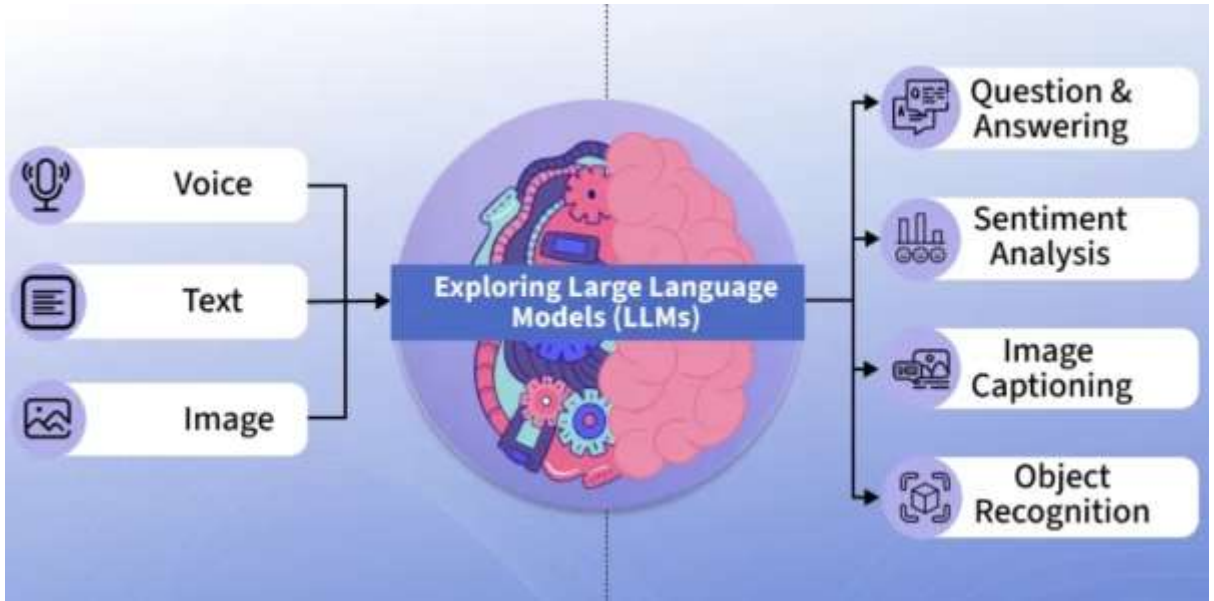
1. **Text Generation:** AI-powered chatbots, content creation, and summarization (e.g., GPT models).
2. **Image and Video Synthesis:** AI-generated art and creative design (e.g., DALL·E, Stable Diffusion).
3. **Music and Audio Production:** AI-composed music and voice cloning.
4. **Code Generation:** Automated coding and debugging assistance (e.g., GitHub Copilot).
5. **Healthcare and Drug Discovery:** AI-generated molecular structures for medical research.

What is a Large Language Model (LLM)

Large Language Models (LLMs) are advanced AI systems built on deep neural networks designed to process, understand and generate human-like text.

By using massive datasets and billions of parameters, LLMs have transformed the way humans interact with technology. It learns patterns, grammar and context from text and can answer questions, write content, translate languages and many more.

- Mordern LLMs include ChatGPT (OpenAI), Google Gemini, Anthropic Claude, etc



Popular LLMs

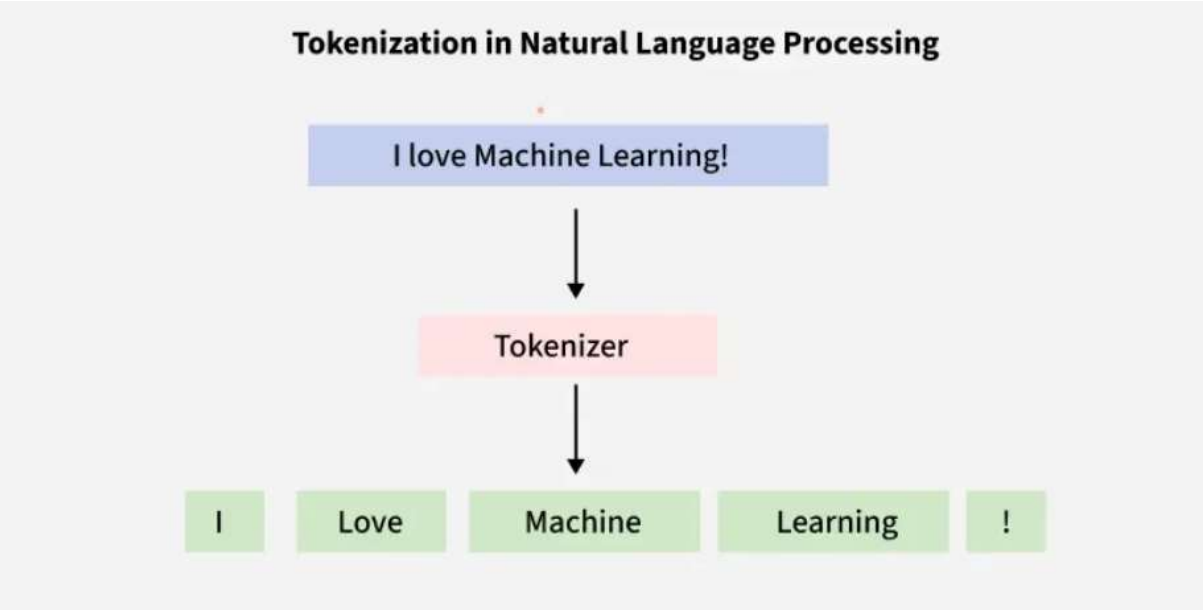
Some of the most widely used LLMs include:

- **GPT-4 and GPT-4o (OpenAI)**: Advanced multimodal reasoning and dialogue capabilities.
- **Gemini 1.5 (Google DeepMind)**: Long-context reasoning, capable of handling 1M+ tokens.
- **Claude 3 (Anthropic)**: Safety-focused, strong at reasoning and summarization.
- **LLaMA 3 (Meta)**: Open-weight model, popular in research and startups.
- **Mistral 7B / Mixtral (Mistral AI)**: Efficient open-source alternatives for developers.
- **BERT and RoBERTa (Google/Facebook)**: Strong embedding models for NLP tasks.
- **mBERT and XLM-R**: Early multilingual LLMs.
- **BLOOM**: Large open-source multilingual model, collaboratively developed.

What is Tokenization?

Tokenization is a fundamental process in Natural Language Processing (NLP) that involves breaking down a stream of text into smaller units called tokens.

These tokens can range from individual characters to full words or phrases, Based on how detailed it needs to be. By converting text into these manageable chunks, machines can more effectively analyze and understand human language.



Tokenization Explained

Tokenization can be likened to teaching someone a new language by starting with the alphabet, then moving on to syllables, and finally to complete words and sentences. This process allows for the dissection of text into parts that are easier for machines to process. For example, consider the sentence, "Chatbots are helpful." When tokenized by words, it becomes:

```
["Chatbots", "are", "helpful"]
```

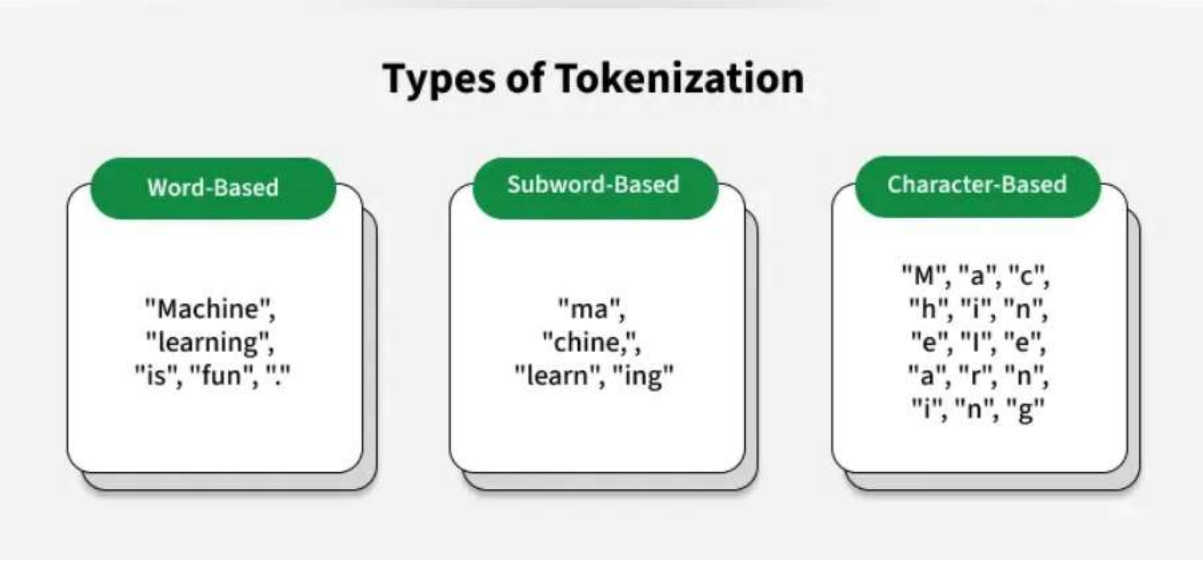
If tokenized by characters, it becomes:

```
["C", "h", "a", "t", "b", "o", "t", "s", " ", "a", "r", "e", " ", "h", "e", "l", "p", "f", "u", "l"]
```

Each approach has its own advantages depending on the context and the specific NLP task at hand.

Types of Tokenization

Tokenization can be classified into several types based on how the text is segmented. Here are some types of tokenization:



1. **Word Tokenization**: This is the most common method where text is divided into individual words. It works well for languages with clear word boundaries, like English.

2. **Character Tokenization**: In this method, text is split into individual characters. This is particularly useful for languages without clear word boundaries or for tasks that require a detailed analysis, such as spelling correction.

3. **Sub-word Tokenization**: Sub-word tokenization strikes a balance between word and character tokenization by breaking down text into units that are larger than a single character but smaller than a full word.

4. **Sentence Tokenization**: Sentence tokenization is also a common technique used to make a division of paragraphs or large set of sentences into separated sentences as tokens.

5. **N-gram Tokenization**: N-gram tokenization splits words into fixed-sized chunks (size = n) of data.

Tokenization Use Cases

Tokenization is critical in numerous applications, including:

- **Information Retrieval:** Tokenization is essential for indexing and searching in systems that store and retrieve information efficiently based on words or phrases.

- **Search Engines**: Use tokenization to process and understand user queries. By breaking down a query into tokens, enhance efficiency match and return precise search results.
- **Machine Translation**: Tools like Google Translate rely on tokenization to convert sentences from one language into another. Segment and Reconstruct to preserve meaning.
- **Speech Recognition**: Voice assistants such as Siri and Alexa use tokenization to process spoken language. Command is first converted into text and then tokenized, enabling the system to understand and execute it accurately.

Implementing Tokenization

Several tools and libraries are available to implement tokenization effectively:

2. NLTK (Natural Language Toolkit): A comprehensive Python library that offers word and sentence tokenization. It's suitable for a wide range of linguistic tasks.
3. SpaCy: A modern and efficient NLP library in Python, known for its speed and support for multiple languages. It is ideal for large-scale applications.

3. BERT Tokenizer: Emerging from the BERT pre-trained model, this tokenizer is context-aware and adept at handling the nuances of language, making it suitable for advanced NLP projects.

4. Byte-Pair Encoding (BPE): An adaptive method that tokenizes based on the most frequent byte pairs in a text. It is effective for languages that combine smaller units to form meaning.

5. Sentence Piece: An unsupervised text tokenizer and de-tokenizer, particularly useful for neural network-based text generation tasks. It supports multiple languages and can tokenize text into sub-words.

Byte-Pair Encoding (BPE) in NLP

- **Byte-Pair Encoding (BPE)** is a text tokenization technique in Natural Language Processing.

- It breaks down words into smaller, meaningful pieces called subwords. It works by repeatedly finding the most common pairs of characters in the text and combining them into a new subword until the vocabulary reaches a desired size.

- This technique helps in handling rare or unknown words by breaking them into smaller parts that the model has already learned during training.

- By reducing the vocabulary size, it makes it easier to work with large amounts of text while allowing the model to understand wide variety of languages

Concepts related to BPE

To understand BPE better, it’s important to know its key concepts:

1. **Vocabulary:** In BPE vocabulary refers to the set of subword units (tokens) used to represent all the words in the corpus. After applying BPE, vocabulary consists of all the subwords that can be used to represent a word in the dataset.

2. **Byte:** It is a basic unit of digital information that consists of 8 bits. It is used to represent characters that are encoded as bytes that will be merged.

- **Character:** It is a single letter, number or punctuation mark in text. We start by treating each word as a sequence of characters and merging process creates subword units based on character pairs.
- **Frequency:** It refers to how many times a particular byte or character appears in the corpus. BPE is driven by the frequency of character pairs meaning that the most frequent pair of characters will be merged first.
- **Merge:** It is the process of combining two consecutive characters or subword units into a new unit. Each merge reduces the total number of tokens in the corpus and leads to more abstract representations of words

How Byte Pair Encoding Works: Step-by-Step

1. Initialize the Vocabulary

Context:

The first step in BPE is to break down your entire text corpus into its **smallest building blocks,** individual characters. This granular approach ensures that every possible word, even those not seen during training, can be represented using the available vocabulary.

Process:

- List every **unique** character found in your dataset (**e.g., a-z, punctuation, spaces**).
- For each word, **split** it into its constituent characters.
- Append a special **end-of-word marker (eg "</w>" or "_")** to each word. This marker helps the algorithm distinguish between words and prevents merges across word boundaries.

Example:

Suppose your dataset contains the words:

- "lower" → l o w e r</w>
- "lowest" → l o w e s t</w>
- "newest" → n e w e s t</w>

Why the end-of-word marker?

It ensures that merges only happen within words, not across them, preserving word boundaries and meaning.

Meet Qwen3 Coder—the open-source MoE powerhouse built for long contexts, smarter coding, and scalable multi-step code mastery.

2. Count Symbol Pairs

Context:

Now, the algorithm looks for **patterns** specifically, pairs of adjacent symbols (characters or previously merged subwords) within each word. By counting how often each pair appears, BPE identifies which combinations are most common and thus most useful to merge.

Process:

- For every word, list all adjacent symbol pairs.
- Tally the frequency of each pair across the entire dataset.

Example:

For “lower” (l o w e r), the pairs are:

- (l, o), (o, w), (w, e), (e, r), (r,)

For “lowest” (l o w e s t):

- (l, o), (o, w), (w, e), (e, s), (s, t), (t,)

For “newest” (n e w e s t):

- (n, e), (e, w), (w, e), (e, s), (s, t), (t,)

Frequency Table Example:

3. Merge the Most Frequent Pair

Context:

The heart of BPE is **merging**. By combining the most frequent pair into a new symbol, the algorithm creates subword units that capture common patterns in the language.

Process:

- Identify the pair with the highest frequency.
- Merge this pair everywhere it appears in the dataset, treating it as a single symbol in future iterations.

Example:

Suppose (w, e) is the most frequent pair (appearing 3 times).

- Merge “w e” into “we”.

Update the words:

- “lower” → l o we r
- “lowest” → l o we s t
- “newest” → n e we s t

Note:

After each merge, the vocabulary grows to include the new subword (“we” in this case).

Decode the core of transformers. Discover how self-attention and multi-head focus transformed NLP forever.

4. Repeat the Process

Context:

BPE is an **iterative** algorithm. After each merge, the dataset changes, and new frequent pairs may emerge.

The process continues until a stopping criterion is met, usually a target vocabulary size or a set number of merges.

Process:

- Recount all adjacent symbol pairs in the updated dataset.
- Merge the next most frequent pair.
- Update all words accordingly.

Example:

If (o, we) is now the most frequent pair, merge it to “owe”:

- “lower” → l owe r
- “lowest” → l owe s t

Continue merging:

- “lower” → low er
- “lowest” → low est
- “newest” → new est

Iteration Table Example:

5. Build the Final Vocabulary

Context:

After the desired number of merges, the vocabulary contains both individual characters and frequently occurring subword units. This vocabulary is used to tokenize any input text, allowing the model to represent rare or unseen words as sequences of known subwords.

Process:

The final vocabulary includes all original characters plus all merged subwords

- Any word can be broken down into a sequence of these subwords, ensuring robust handling of out-of-vocabulary terms.

Example:

Final vocabulary might include:

{l, o, w, e, r, s, t, n, we, owe, low, est, new, lower, lowest, newest, }

Tokenization Example:

- “lower” → lower
- “lowest” → lowest
- “newest” → newest

What Are Word Embeddings?

- A word embedding is a learned representation for text where words that have the same meaning have a similar representation.
- It is this approach to representing words and documents that may be considered one of the key breakthroughs of deep learning on challenging natural language processing problems.
- Word embeddings are in fact a class of techniques where individual words are represented as real-valued vectors in a predefined vector space.

- Each word is mapped to one vector and the vector values are learned in a way that resembles a neural network, and hence the technique is often lumped into the field of deep learning.
- Key to the approach is the idea of using a dense distributed representation for each word.
- Each word is represented by a real-valued vector, often tens or hundreds of dimensions.

- This is contrasted to the thousands or millions of dimensions required for sparse word representations, such as a one-hot encoding.

Word Embedding Algorithms

- Word embedding methods learn a real-valued vector representation for a predefined fixed sized vocabulary from a corpus of text.

- The learning process is either joint with the neural network model on some task, such as document classification, or is an unsupervised process, using document statistics.

- This section reviews three techniques that can be used to learn a word embedding from text data

1. Embedding Layer

- An embedding layer, for lack of a better name, is a word embedding that is learned jointly with a neural network model on a specific natural language processing task, such as language modelling or document classification.
- It requires that document text be cleaned and prepared such that each word is one-hot encoded. The size of the vector space is specified as part of the model, such as 50, 100, or 300 dimensions.

- The vectors are initialized with small random numbers. The embedding layer is used on the front end of a neural network and is fit in a supervised way using the Backpropagation algorithm

… when the input to a neural network contains symbolic categorical features (e.g. features that take one of k distinct symbols, such as words from a closed vocabulary), it is common to associate each possible feature value (i.e., each word in the vocabulary) with a d-dimensional vector for some d. These vectors are then considered parameters of the model, and are trained jointly with the other parameters.

- The one-hot encoded words are mapped to the word vectors. If a multilayer Perceptron model is used, then the word vectors are concatenated before being fed as input to the model. If a recurrent neural network is used, then each word may be taken as one input in a sequence.
- This approach of learning an embedding layer requires a lot of training data and can be slow, but will learn an embedding both targeted to the specific text data and the NLP task.

2. Word2Vec

Word2Vec is a statistical method for efficiently learning a standalone word embedding from a text corpus.

It was developed by Tomas Mikolov, et al. at Google in 2013 as a response to make the neural-network-based training of the embedding more efficient and since then has become the de facto standard for developing pre-trained word embedding.

Additionally, the work involved analysis of the learned vectors and the exploration of vector math on the representations of words. For example, that subtracting the "*man-ness*" from "*King*" and adding "*women-ness*" results in the word "*Queen*", capturing the analogy "*king is to queen as man is to woman*".

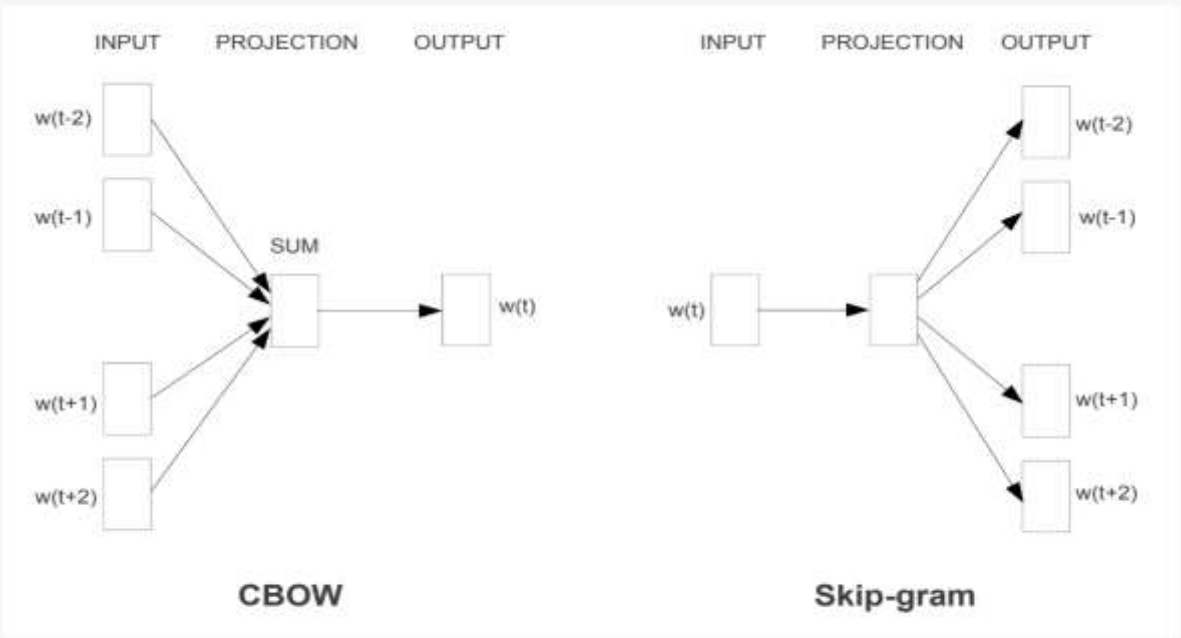
We find that these representations are surprisingly good at capturing syntactic and semantic regularities in language, and that each relationship is characterized by a relation-specific vector offset. This allows vector-oriented reasoning based on the offsets between words. For example, the male/female relationship is automatically learned, and with the induced vector representations, "King – Man + Woman" results in a vector very close to "Queen."

Two different learning models were introduced that can be used as part of the word2vec approach to learn the word embedding; they are:

- Continuous Bag-of-Words, or CBOW model.
- Continuous Skip-Gram Model.

The CBOW model learns the embedding by predicting the current word based on its context. The continuous skip-gram model learns by predicting the surrounding words given a current word.

The continuous skip-gram model learns by predicting the surrounding words given a current word.



Word2Vec Training Models
Taken from "Efficient Estimation of Word Representations in Vector Space", 2013

Both models are focused on learning about words given their local usage context, where the context is defined by a window of neighbouring words. This window is a configurable parameter of the model.

The size of the sliding window has a strong effect on the resulting vector similarities. Large windows tend to produce more topical similarities […], while smaller windows tend to produce more functional and syntactic similarities

3. GloVe

The Global Vectors for Word Representation, or GloVe, algorithm is an extension to the word2vec method for efficiently learning word vectors, developed by Pennington, et al. at Stanford.

Classical vector space model representations of words were developed using matrix factorization techniques such as Latent Semantic Analysis (LSA) that do a good job of using global text statistics but are not as good as the learned methods like word2vec at capturing meaning and demonstrating it on tasks like calculating analogies (e.g. the King and Queen example above).

Summarize
Glove
Global
Vector

Summarize

Glove

Global Vector

Derive the semantic relationship between words using word-word co-occurrence matrix

I love Learnerea

I love DataScience

Window = 1

	I	love	Learnerea	DataScience
I				
love				
Learnerea				
DataScience				

	I	love	Learnerea	DataScience
I	0	2	0	0
love	2	0	1	1
Learnerea	0	1	0	0
DataScience	0	1	0	0

i

j

X

X_{ij}

	I	love	Learnerea	DataScience
I	0	2	0	0
love	2	0	1	1
Learnerea	0	1	0	0
DataScience	0	1	0	0

i

j

I love Learnerea

I love DataScience

$$P_{ij} = P(W_i | W_j) = \frac{X_{ij}}{X_j}$$

$$P(I | Love) = \frac{2}{2} = 1$$

GloVe is an approach to marry both the global statistics of matrix factorization techniques like LSA with the local context-based learning in word2vec.

Rather than using a window to define local context, GloVe constructs an explicit word-context or word co-occurrence matrix using statistics across the whole text corpus. The result is a learning model that may result in generally better word embeddings.

GloVe, is a new global log-bilinear regression model for the unsupervised learning of word representations that outperforms other models on word analogy, word similarity, and named entity recognition tasks.

What are transformers in artificial intelligence?

Transformers are a type of neural network architecture that transforms or changes an input sequence into an output sequence. They do this by learning context and tracking relationships between sequence components. For example, consider this input sequence: "What is the color of the sky?" The transformer model uses an internal mathematical representation that identifies the relevancy and relationship between the words color, sky, and blue. It uses that knowledge to generate the output: "The sky is blue."

Organizations use transformer models for all types of sequence conversions, from speech recognition to machine translation and protein sequence analysis.

Transformer is a neural network architecture that has fundamentally changed the approach to Artificial Intelligence. Transformer was first introduced in the seminal paper "Attention is All You Need" in 2017 and has since become the go-to architecture for deep learning models, powering text-generative models like OpenAI's **GPT**, Meta's **Llama**, and Google's **Gemini**. Beyond text, Transformer is also applied in audio generation, image recognition, protein structure prediction, and even game playing, demonstrating its versatility across numerous domains.

What are the use cases for transformers?

You can train large transformer models on any sequential data like human languages, music compositions, programming languages, and more. The following are some examples of use cases.

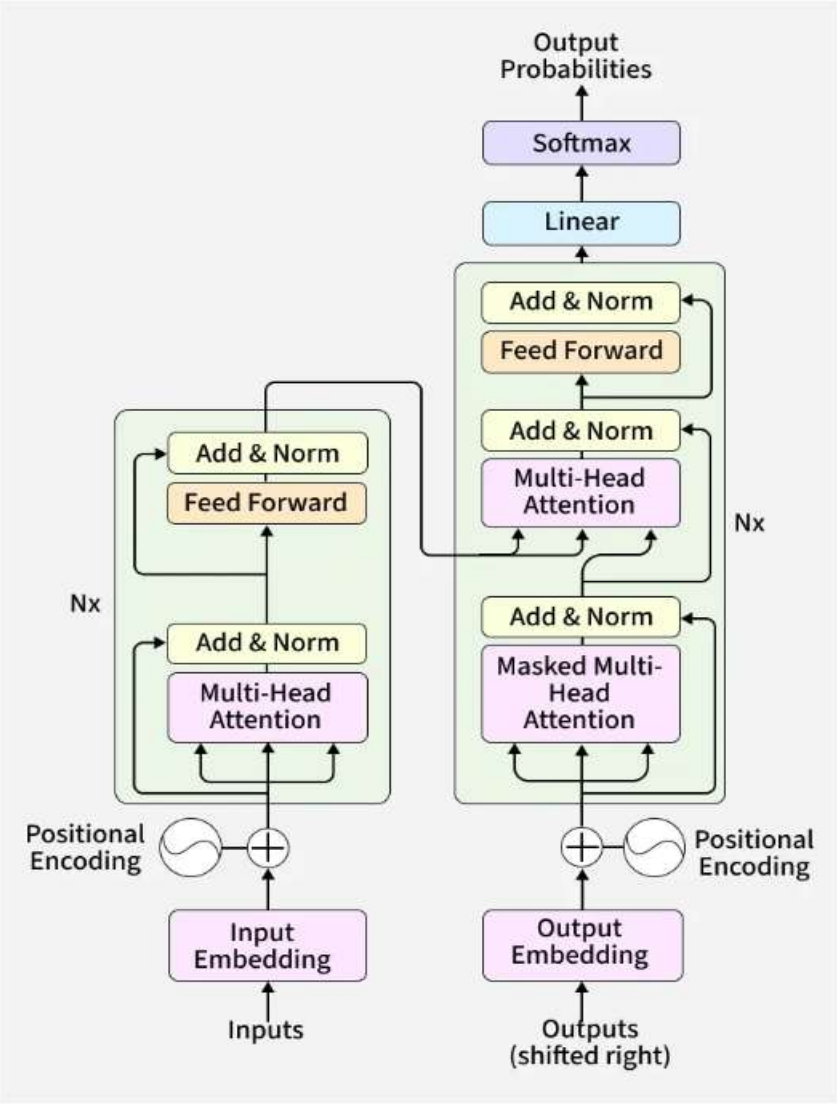
Natural language processing

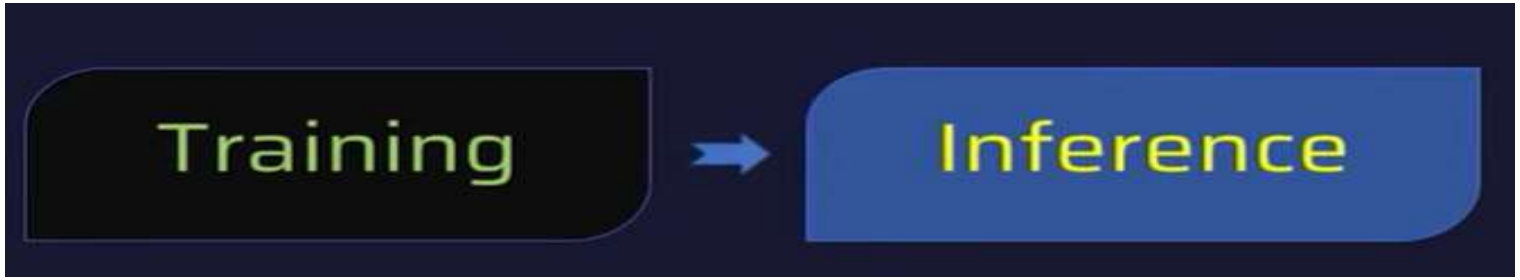
Machine translation

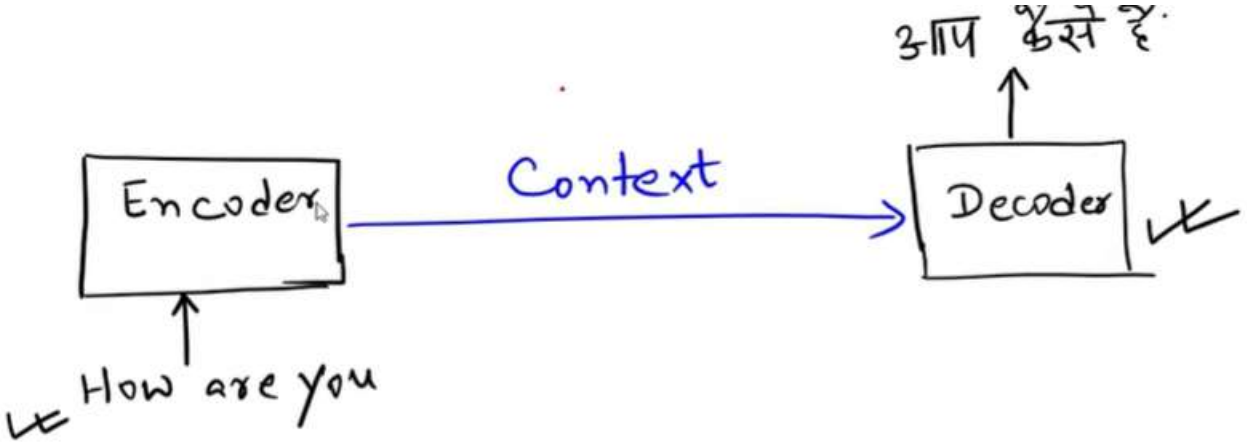
DNA sequence analysis

Protein structure analysis

What are the components of transformer architecture?




Training
Inference



Eng	Hindi
How are you	आप कैसे हैं

Encoder Steps:

Tokenization - 0
Embedding - 1
Positional Encoding - 2
Multi-head Attention - 3
Add & LayerNorm - 4
Feedforward Network - 5
Add & LayerNorm - 6

Decoder Steps:

Shifted Input + Embedding + Positional Encoding - 7
Masked Self-Attention - 8
Add & LayerNorm -9
Cross Attention (with encoder output) - 10
Add & LayerNorm - 11
Feedforward - 12
Add & LayerNorm - 13
Final Linear Projection + Softmax - 14

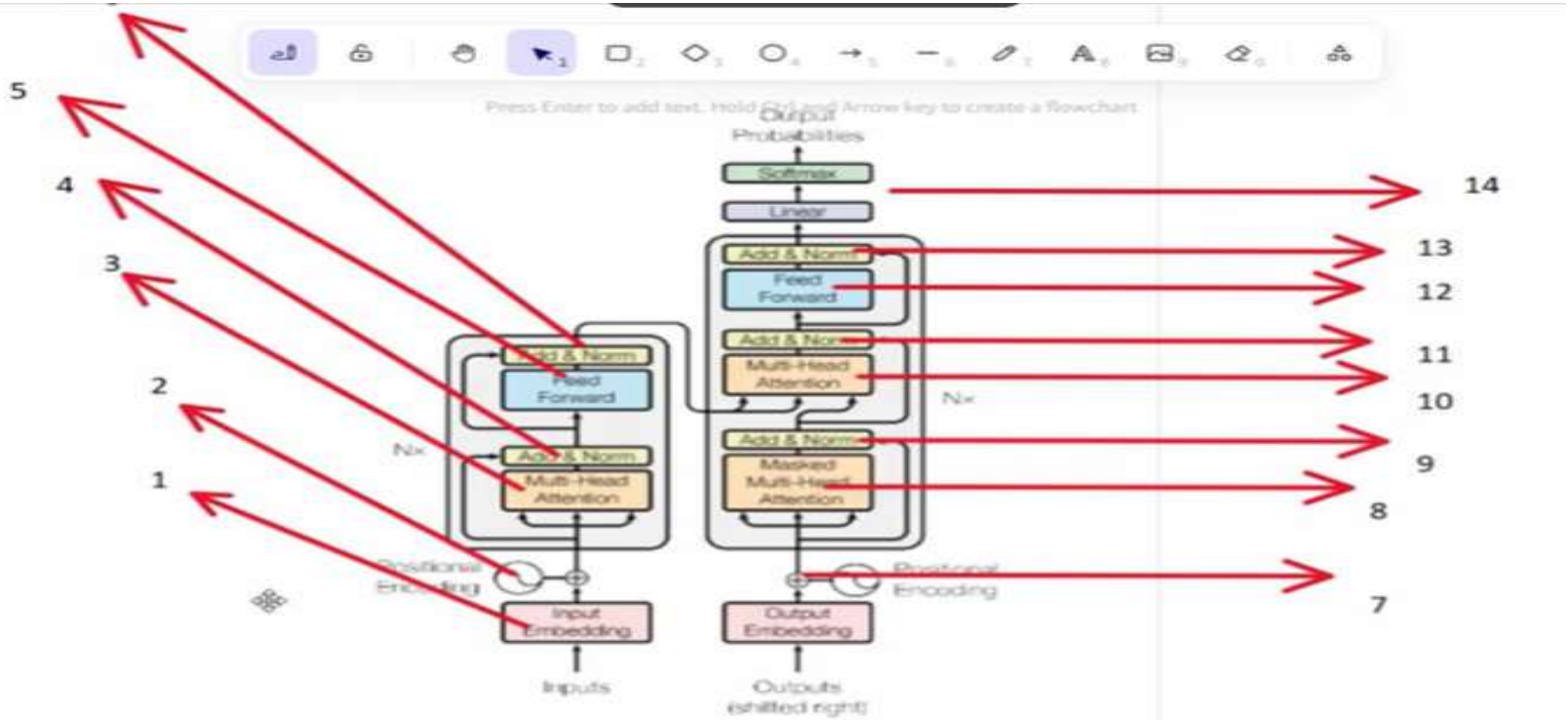


Figure 1: The Transformer - model architecture.

Press Escape or Ctrl+Enter to finish editing

Encoder Steps

Step 0

How are you => "how" "are" "you"

Positional Encoding in Transformers

Input- my name is Aarohi and I love coding

["my", "name", "is", "Aarohi", "and", "I", "love", "coding"]

[101, 223, 31, 987, 45, 12, 34, 556]

"my" = [0.2, -0.7, 0.1, ...]

"name" = [0.5, 0.8, -0.67, ...]

"is" = [0.05, 0.08, -0.04, ...]

"Aarohi" = [0.35, 0.38, -0.33, ...]

"and" = [0.12, -0.17, 0.14, ...]

"I" = [0.52, 0.58, -0.67, ...]

"love" = [0.06, 0.01, -0.14, ...]

"coding" = [0.25, 0.28, -0.32, ...]

3.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [9].

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

We also experimented with using learned positional embeddings [9] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

Input- my name is Aarohi and

["my", "name", "is", "Aarohi", "

[101, 223, 31, 987, 4

$pos=$ $i=$ $d_{\text{model}}=$

What is the length of an embedding vector?

d_{model} gpt-3 4096

d_model = 256, 512

sentence :

[

e_my, 4096

e_name, 4096

e_is, "

e_Aarohi, "

e_and, "

e_I, "

e_love, "

e_coding "

]

Input- my name is Aarohi and I love coding

["my", "name", "is", "Aarohi", "and", "I", "love", "coding"]

[101, 223, 31, 987, 45, 12, 34, 556]

my [- - - - - - - - - - - -] 512

name [- - - - - - - - - - - -] 512

is [- - - - - - - - - - - -]

Model Name
GPT-3 Small
GPT-3 Medium
GPT-3 Large
GPT-3 XL
GPT-3 2.7B
GPT-3 6.7B
GPT-3 13B
GPT-3 175B or

Token	Word	pos
1	my	0
2	name	1
3	is	2
4	Aarohi	3
5	and	4
6	I	5
7	love	6
8	coding	7

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

- Input- my name is Aarohi and I love coding
  ["my", "name", "is", "Aarohi", "and", "I", "love", "coding"]

🔎 How Multi-Head Attention Works

1. Self-Attention (the foundation)

- Each input token is transformed into three vectors:
  - **Query (Q)** – what this token is "asking" about others.
  - **Key (K)** – what this token "offers" for comparison.
  - **Value (V)** – the actual information carried.
- Attention scores are computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- where d_k is the dimension of the key vectors (scaling factor).

Breakdown of Add & Norm

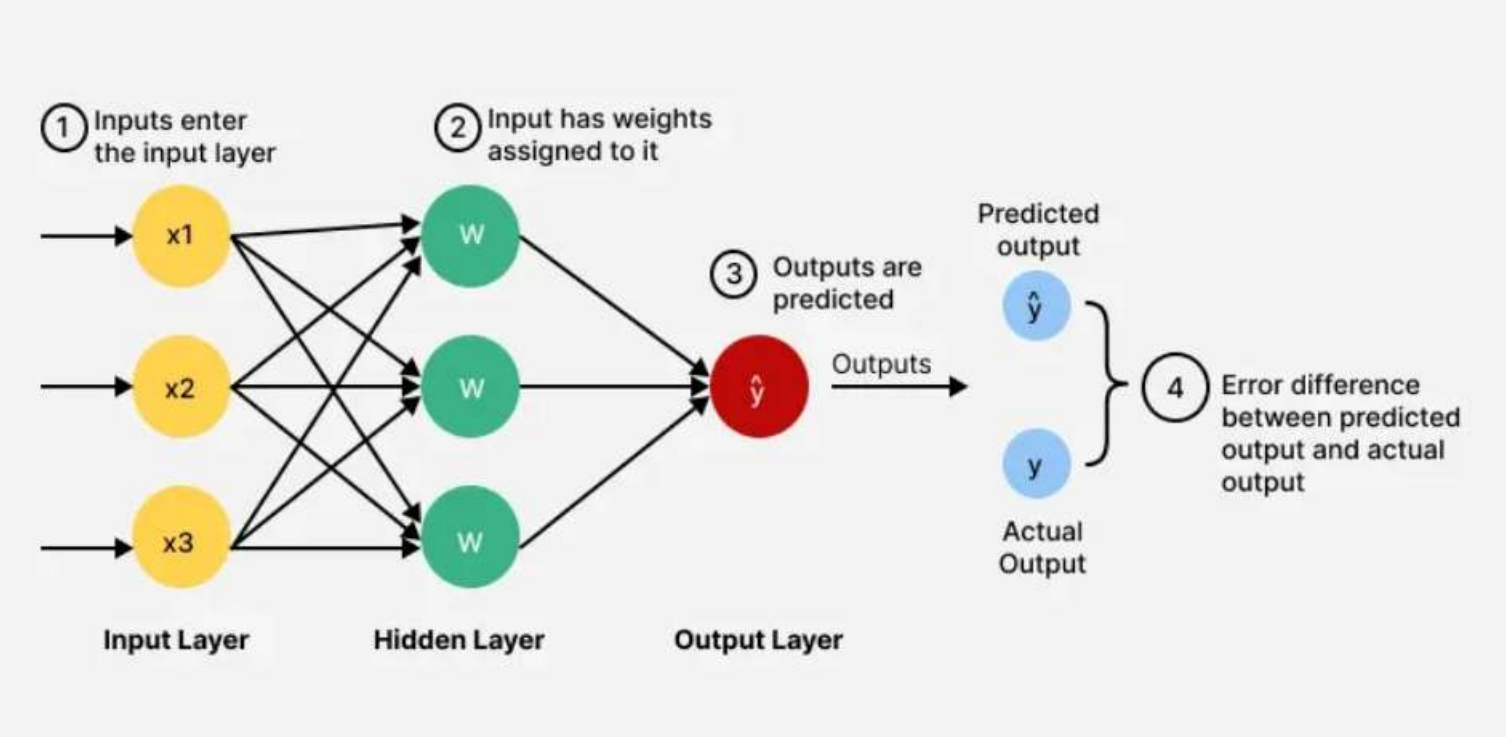
1. Residual Connection (Add)

- The input to a sub-layer is added back to its output:

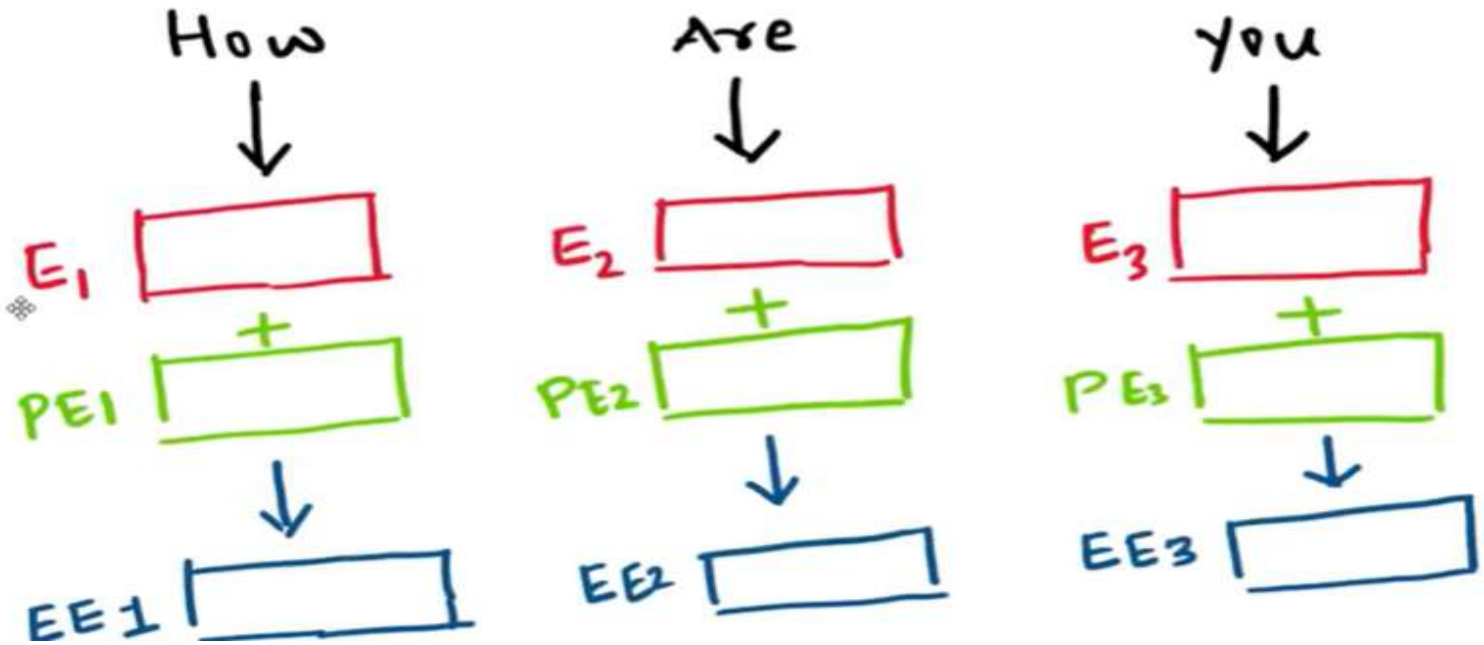
- This ensures that the original information flows forward, even if the sub-layer transformation changes it.
- Benefits:
- Helps avoid vanishing/exploding gradients in deep networks.
- Makes optimization easier by allowing the model to learn modifications rather than completely new transformations.

2. Layer Normalization (Norm)

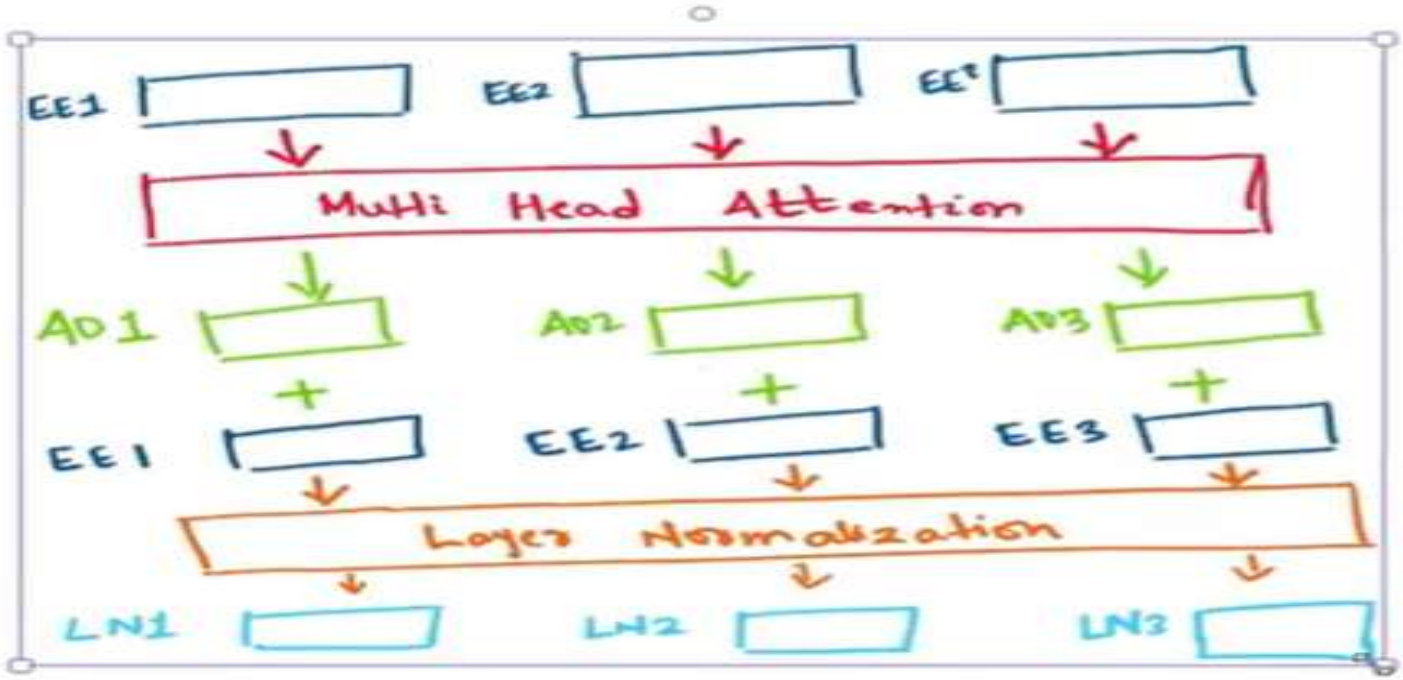
- After the residual addition, the result is normalized across features.
- This stabilizes the distribution of activations, preventing internal covariate shift.
- Benefits:
- Speeds up convergence.
- Improves generalization.
- Keeps training stable even with very deep stacks of layers.


1 Inputs enter the input layer
2 Input has weights assigned to it
3 Outputs are predicted
4 Error difference between predicted output and actual output
x1
x2
x3
W
W
W
ŷ
Outputs
Predicted output
ŷ
y
Actual Output
Input Layer
Hidden Layer
Output Layer

Step 1 and Step 2



Step 3 & 4


EE1
EE2
EE3
Multi Head Attention
AO1
AO2
AO3
+
+
+
EE1
EE2
EE3
Layer Normalization
LN1
LN2
LN3

What is the length of an embedding vector?

Model Name
GPT-3 Small
GPT-3 Medium
GPT-3 Large
GPT-3 XL
GPT-3 2.7B
GPT-3 6.7B
GPT-3 13B
GPT-3 175B or

d_{model} gpt-3 4096

d_model = 256, 512

sentence :

[

e_my, 4096

e_name, 4096

e_is, "

e_Aarohi, "

e_and, "

e_I, "

e_love, "

e_coding "

]

Input- my name is Aarohi and I love coding

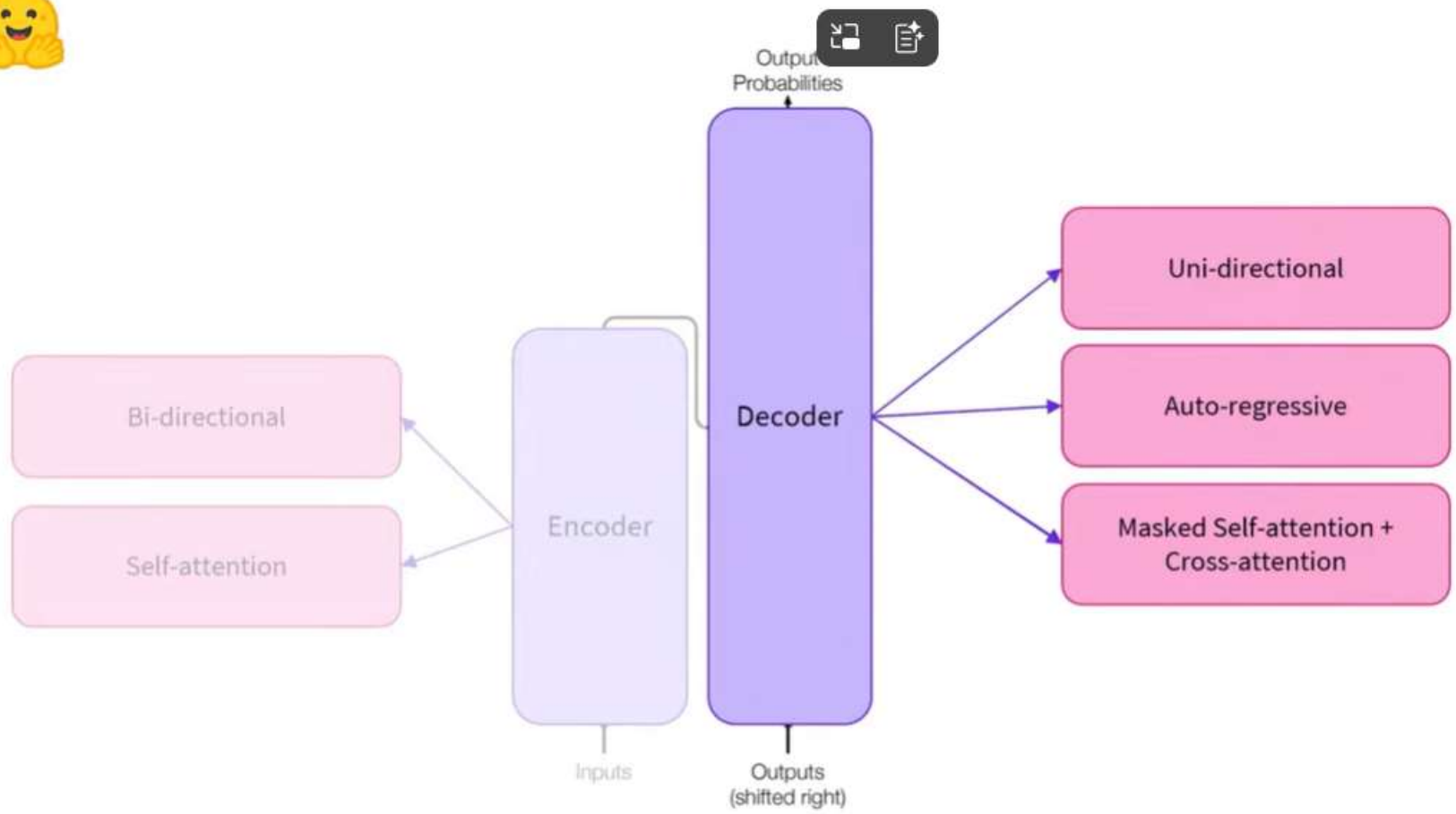
["my", "name", "is", "Aarohi", "and", "I", "love", "coding"]

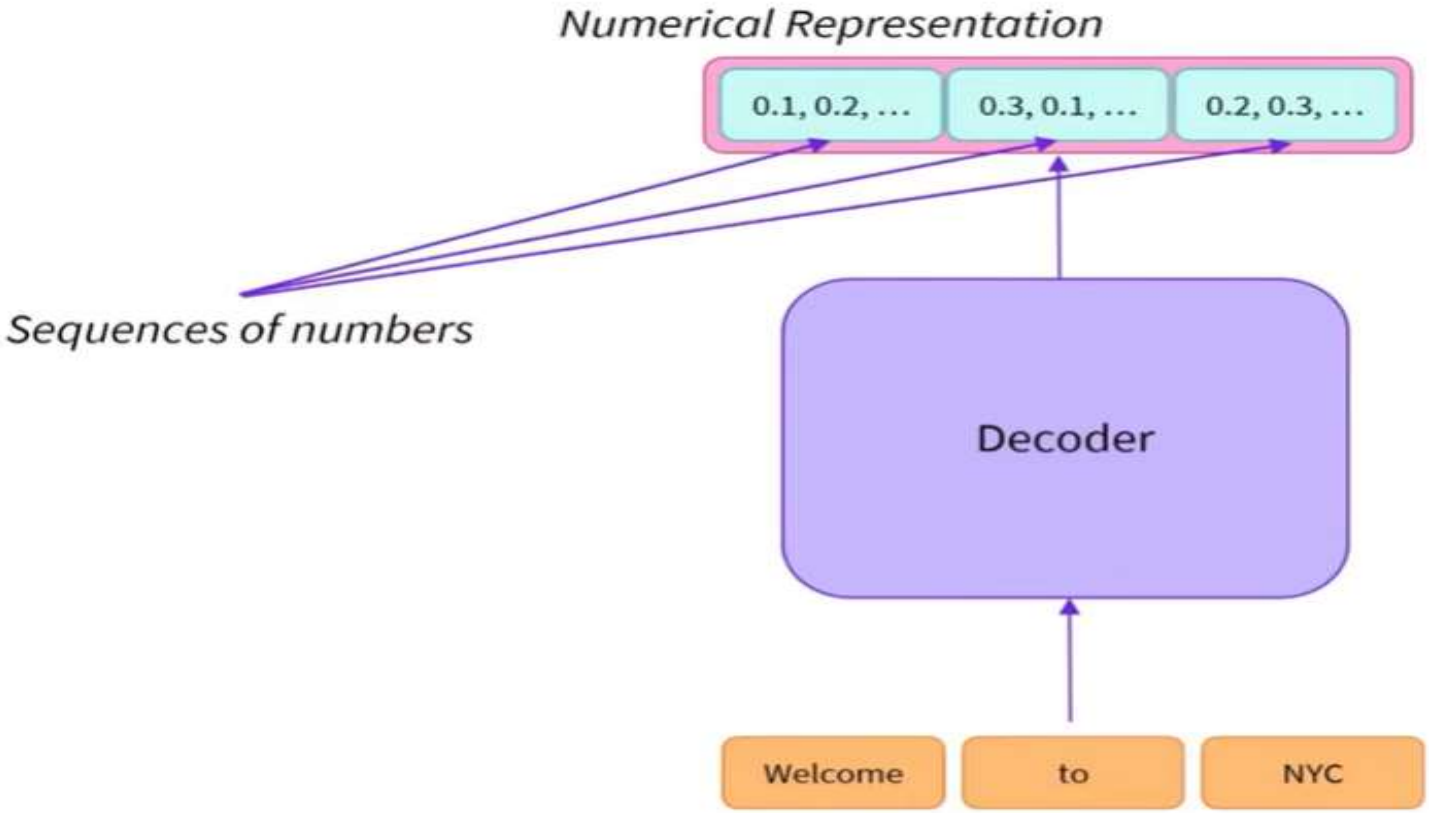
[101, 223, 31, 987, 45, 12, 34, 556]

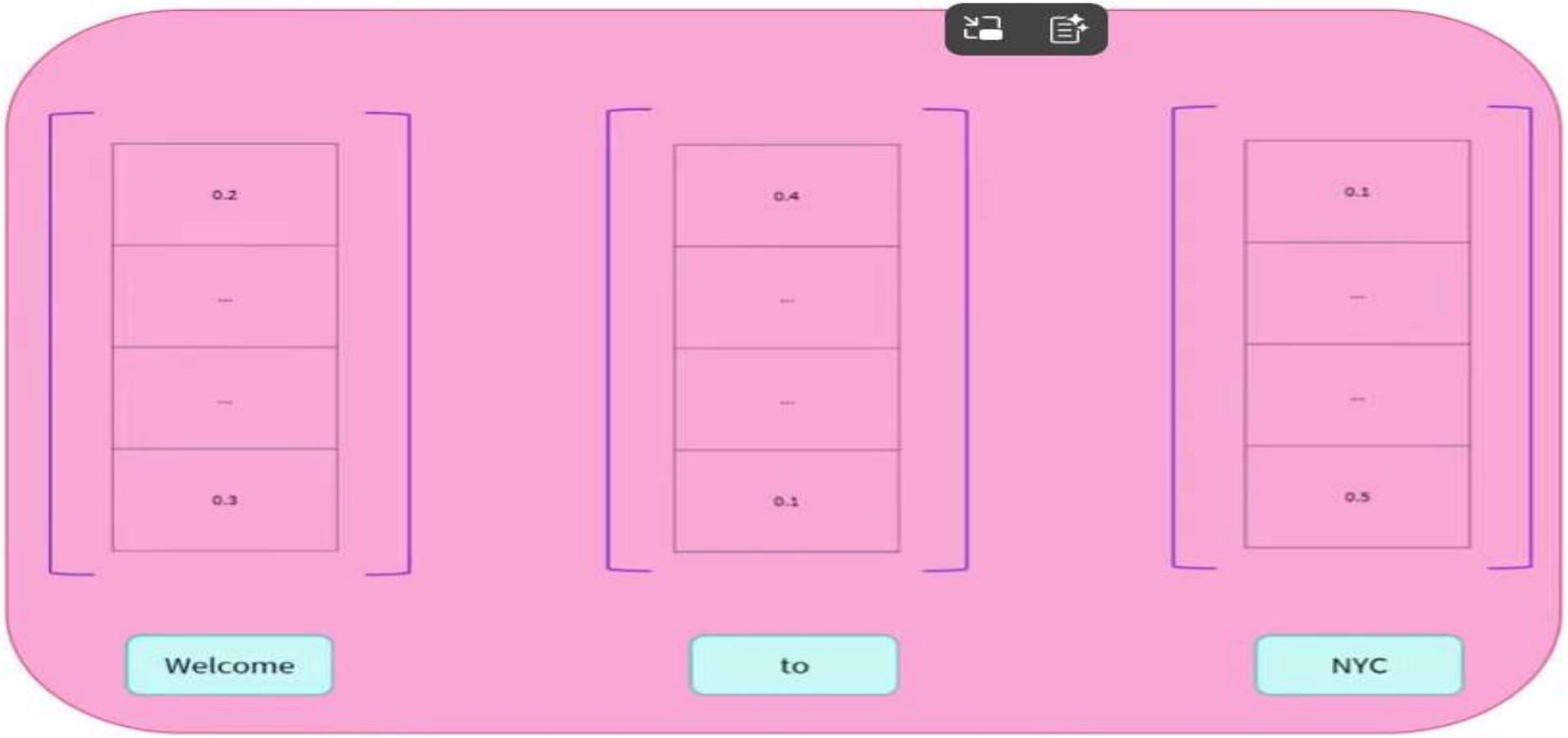
my [- - - - - - - - - - - -] 512

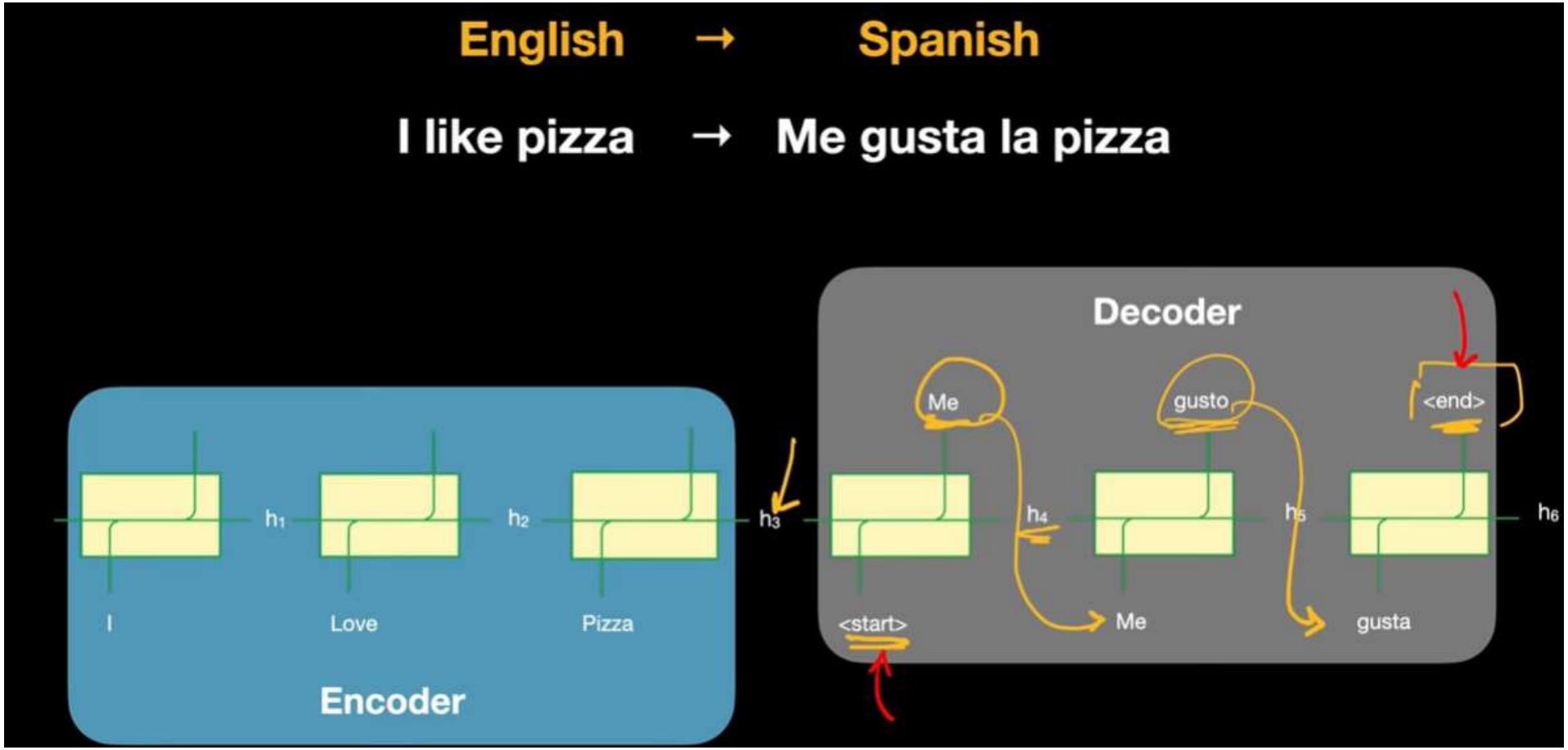
name [- - - - - - - - - - - -] 512

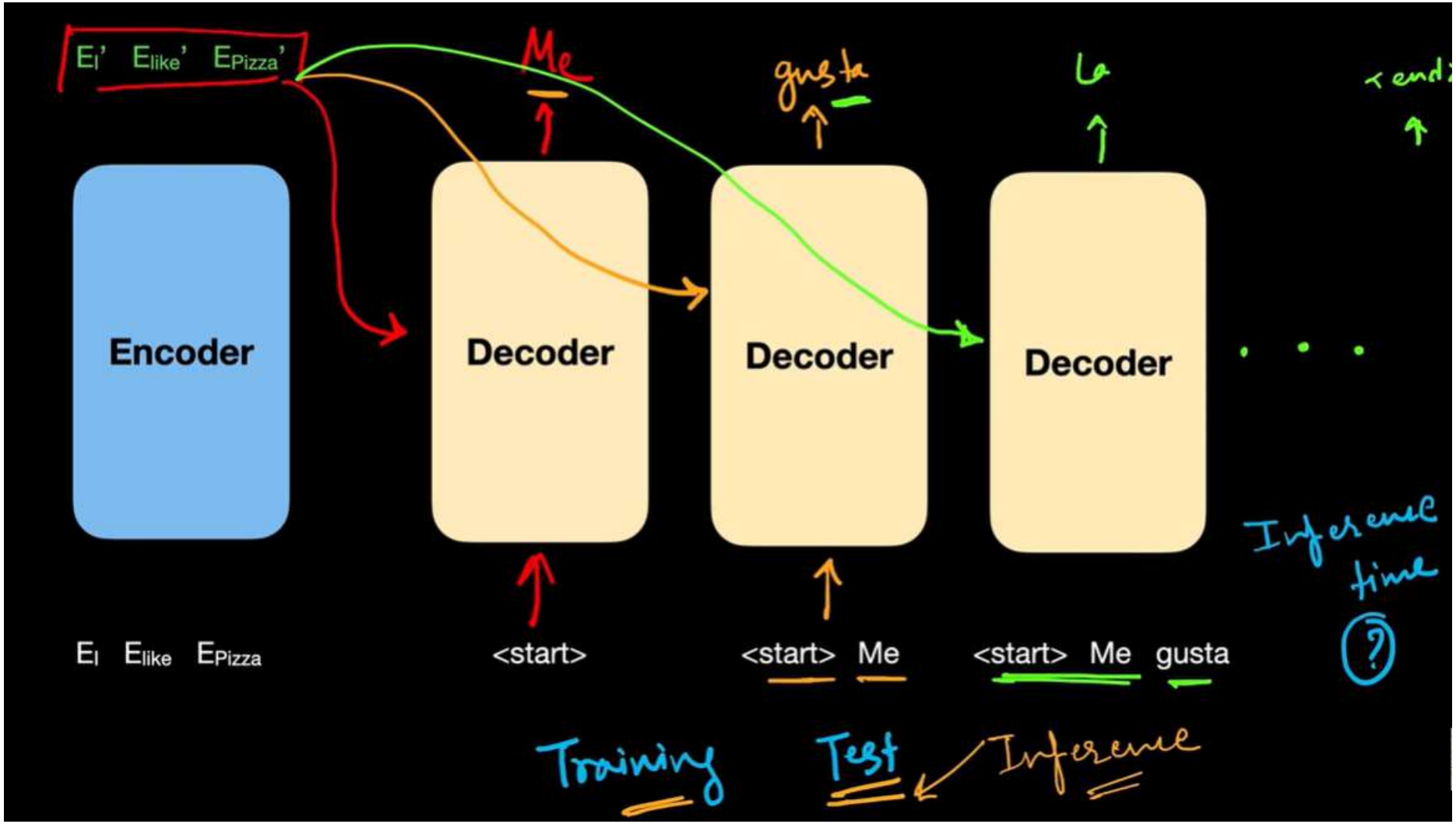
is [- - - - - - - - - - - -]

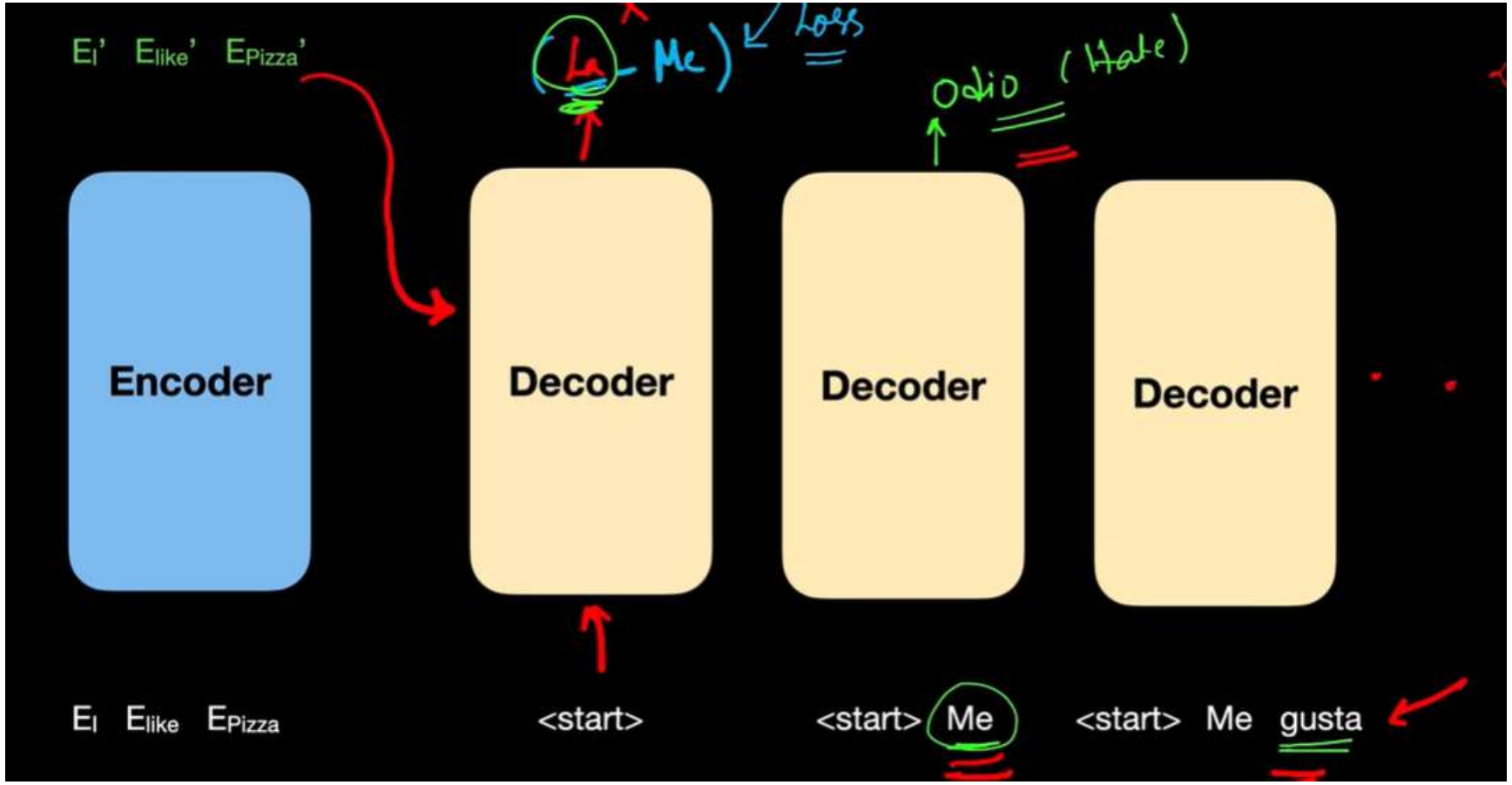

Output
Probabilities
Decoder
Uni-directional
Auto-regressive
Masked Self-attention +
Cross-attention
Bi-directional
Self-attention
Encoder
Inputs
Outputs
(shifted right)

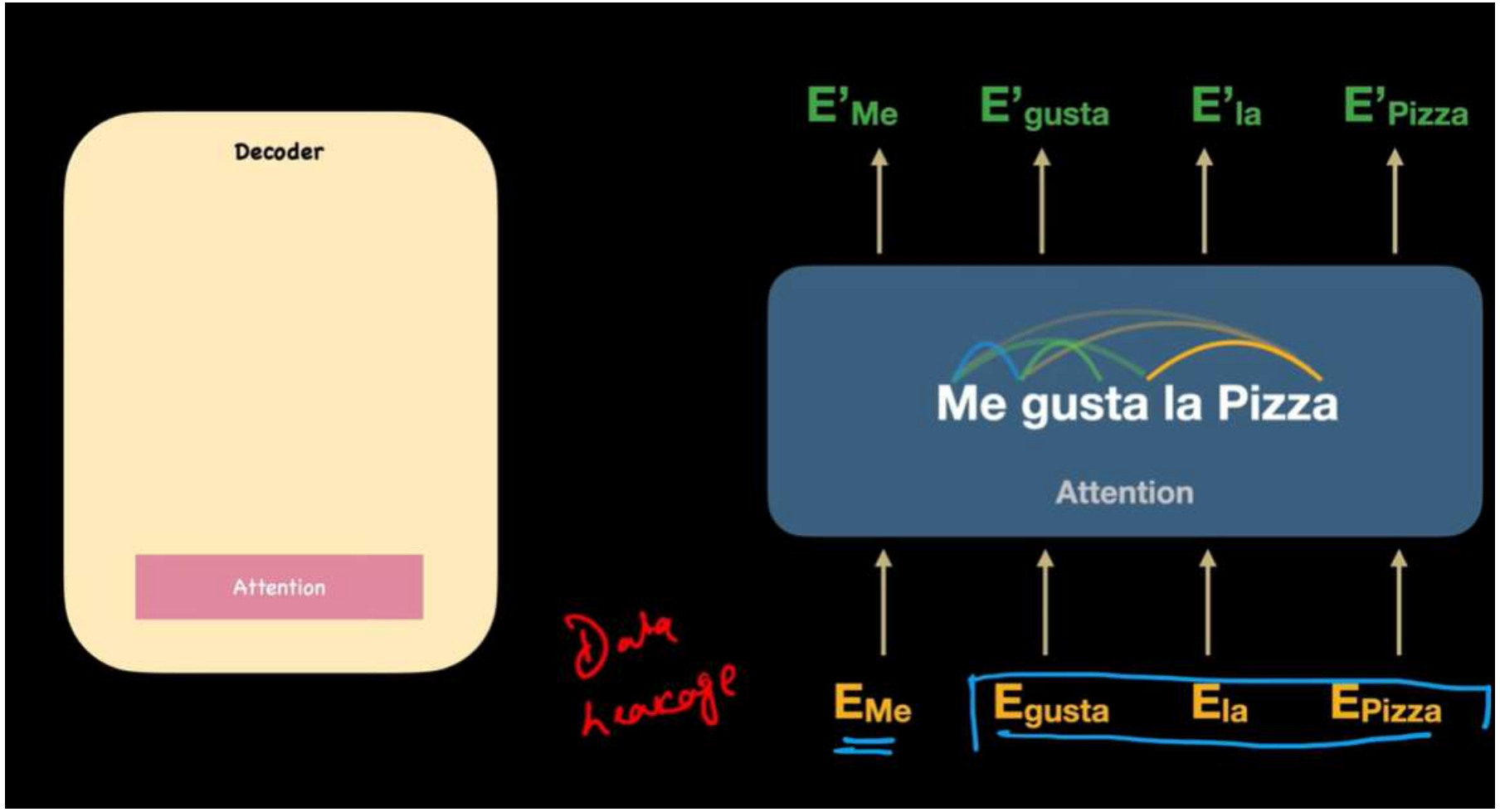

Numerical Representation
0.1, 0.2, ...
0.3, 0.1, ...
0.2, 0.3, ...
Sequences of numbers
Decoder
Welcome
to
NYC


0.2
...
...
0.3
0.4
...
...
0.1
0.1
...
...
0.5
Welcome
to
NYC


English → Spanish
I like pizza → Me gusta la pizza
Decoder
Me
gusto
<end>
h1
h2
h3
h4
h5
h6
I
Love
Pizza
<start>
Me
gusta
Encoder


EI' Elike' EPizza'
Me
gusta
La
<end>
Encoder
Decoder
Decoder
Decoder
. . .
Inference time
?
EI Elike EPizza
<start>
<start> Me
<start> Me gusta
Training
Test
Inference


EI' Elike' EPizza'
Encoder
Decoder
Decoder
Decoder
EI Elike EPizza
<start>
<start> Me
<start> Me gusta


Decoder
Attention
E'Me
E'gusta
E'la
E'Pizza
Me gusta la Pizza
Attention
Data Language
EMe
Egusta
Ela
EPizza

1. Self Attention Mechanism

The self attention mechanism allows transformers to determine which words in a sentence are most relevant to each other. This is done using a scaled dot-product attention approach:

Each word in a sequence is mapped to three vectors:

- **Query (Q)**
- **Key (K)**
- **Value (V)**

Attention scores are computed as: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

These scores determine how much attention each word should pay to others.

2. Multi-Head Attention

Instead of one attention mechanism, transformers use multiple attention heads running in parallel. Each head captures different relationships or patterns in the data, enriching the model's understanding.

3. Positional Encoding

Unlike RNNs, transformers lack an inherent understanding of word order since they process data in parallel. To solve this problem Positional Encodings are added to token embeddings providing information about the position of each token within a sequence.

4. Position-wise Feed-Forward Networks

The Feed-Forward Networks consist of two linear transformations with a ReLU activation. It is applied independently to each position in the sequence.

Mathematically:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

This transformation helps refine the encoded representation at each position.

5. Embeddings

Transformers cannot work with raw words as they need numbers. So, each input token (word or subword) is converted into a vector, called an embedding.

- Both encoder input tokens and decoder input tokens are converted into embeddings.
- These embeddings are trainable, meaning the model learns the best numeric representation for each token.
- The same weight matrix is shared for Encoder embeddings, Decoder embeddings and the final linear layer before softmax
- The embeddings are scaled by model to keep values stable before adding positional encoding.

Embeddings turn words into meaningful numeric vectors that the transformer can process

6. Encoder-Decoder Architecture

The **encoder-decoder** structure is key to transformer models. The encoder processes the input sequence into a vector, while the decoder converts this vector back into a sequence. Each encoder and decoder layer includes self-attention and feed-forward layers.

For example, a French sentence "Je suis étudiant" is translated into "I am a student" in English.

Transformers apply attention in three different places:

1. Encoder Self-Attention

- Q, K, V all come from the encoder’s previous layer.
- Every word can attend to every other word in the input.
- This helps the encoder understand full context (long-range meaning).

2. Decoder Self-Attention (Masked)

- Q, K, V all come from the decoder’s previous layer.
- Future tokens are masked (blocked), so each position only sees previous tokens.
- This keeps decoding auto-regressive i.e the model predicts one word at a time.

3. Encoder–Decoder Attention

- Queries come from the decoder.
- Keys and Values come from the encoder output.
- This lets the decoder look at important parts of the input sentence while generating output.

Together, these three attention types allow the transformer to read the entire input at once and then generate outputs step-by-step with full context.

7. SoftMax Layer for Output Prediction

After the decoder processes the sequence, it must predict the next token.

- The decoder output is passed through a linear layer (whose weights are shared with embeddings).
- Then the softmax function converts these scores into probabilities.
- The token with the highest probability becomes the predicted next word.

Intuition with Example

For instance, in the sentence "The cat didn't chase the mouse, because it was not hungry" the word 'it' refers to 'cat'. The self-attention mechanism helps the model correctly associate 'it' with 'cat' ensuring an accurate understanding of sentence structure.

Applications

Some of the applications of transformers are:

- **NLP Tasks**: Transformers are used for machine translation, text summarization, named entity recognition and sentiment analysis.
- **Speech Recognition**: They process audio signals to convert speech into transcribed text.
- **Computer Vision**: Transformers are applied to image classification, object detection and image generation.
- **Recommendation Systems**: They provide personalized recommendations based on user preferences.

Text and Music Generation: Transformers are used for generating text like articles and composing music

Pre Trained Model

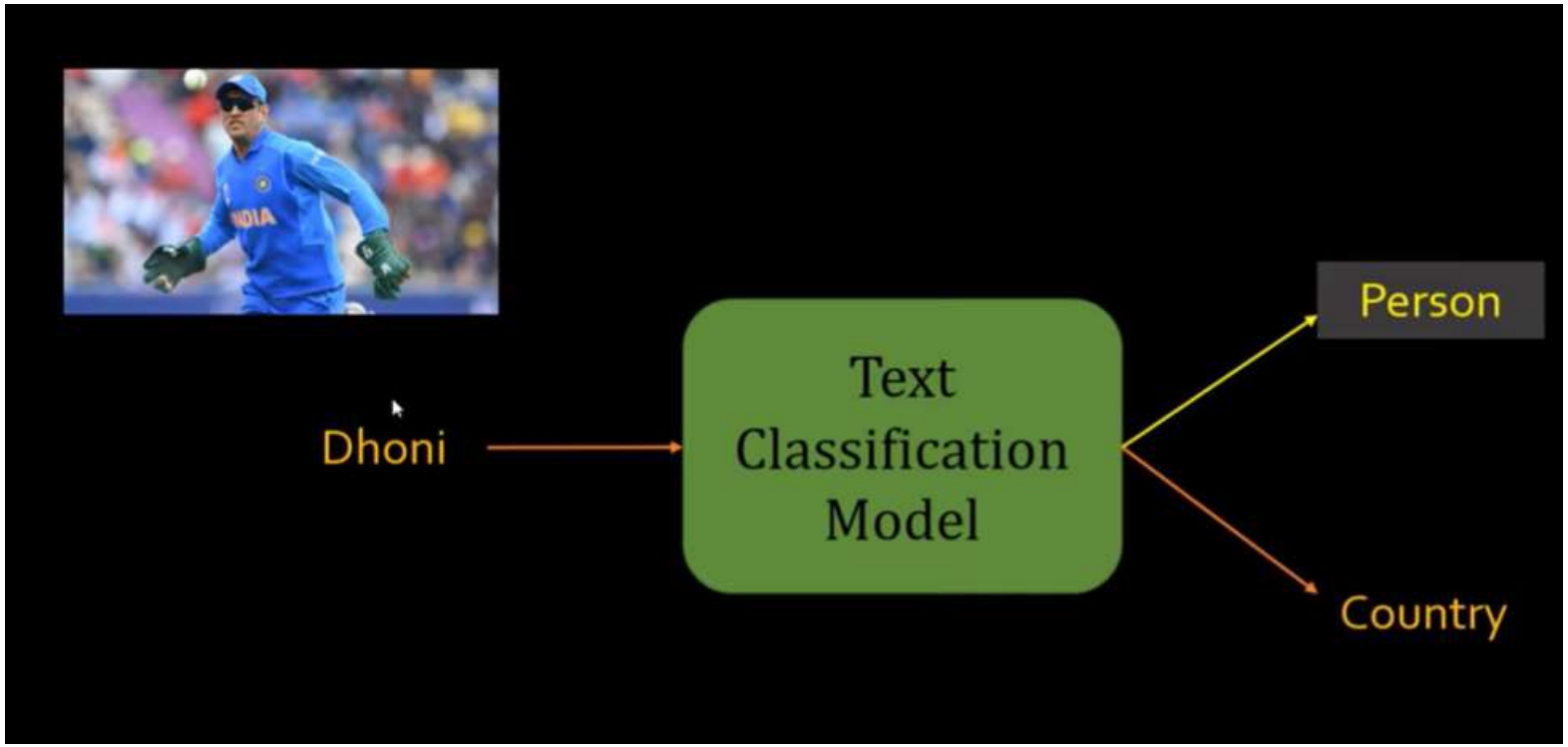
- A pre-trained model is a machine learning (ML) model that has been trained on a large dataset and can be fine-tuned for a specific task.
- Pre-trained models are often used as a starting point for developing ML models, as they provide a set of initial weights and biases that can be fine-tuned for a specific task.
- Pre-trained models are often trained on large, diverse datasets and have been trained to recognize a wide range of patterns and features. As a result, they can provide a strong foundation for fine-tuning and can significantly improve the performance of the model

- Pre-trained models come in a variety of forms, such as language models, object detection models, and picture classification models.
- Convolutional neural networks are frequently used as the foundation for image classification models, which are trained to categorize images into predetermined categories (CNNs).
- CNNs or region-based convolutional neural networks are frequently used as the foundation for object recognition models, which are taught to recognise and categorize items in photos or videos (R-CNNs).
- Recurrent neural networks (RNNs) or transformers are frequently used as the foundation for language models, which are trained to predict the next word in a sequence.
- Overall, pre-trained models are a useful tool in ML, and can be used as a starting point for developing ML models.
- They provide a set of initial weights and biases that can be fine-tuned for a specific task, and can significantly improve the performance of the model.

What is BERT?

BERT (Bidirectional Encoder Representations from Transformers) leverages a transformer-based neural network to understand and generate human-like language. BERT employs an encoder-only architecture.

In the original **Transformer architecture**, there are both encoder and decoder modules. The decision to use an encoder-only architecture in BERT suggests a primary emphasis on understanding input sequences rather than generating output sequences.


Dhoni
Text Classification Model
Person
Country

How can we capture similarities between two words?

Bedrooms: 3
Area: 1850 sqr ft
Bathrooms: 2.5

Bedrooms: 3
Area: 1700 sqr ft
Bathrooms: 2

Bedrooms: 10
Area: 7500 sqr ft
Bathrooms: 2

Dhoni

Person: 1
Healthy/fit: 0.9
Location: 0
Has two eyes: 1
Has government: 0

Cummins

Person: 1
Healthy/fit: 0.87
Location: 0
Has two eyes: 1
Has government: 0

Australia

Person: 0
Healthy/fit: 0.7
Location: 1
Has two eyes: 0
Has government: 1

	ashes	Australia	Bat	Cummins	cover	Dhoni	World cup	..	Zimbabwe
Person	0	0.02	0.1	**0.95**	0.03	**0.96**	0.67	...	0.04
Country	0	**0.97**	0	0	0	0	0	...	**1**
Healthy & Fit	0	0	0.3	**0.87**	0	**0.9**	0	...	0
Event	**1**	0.1	0	0	0.4	0	**1**	...	0
gear	0	0	**1**	0	0	0	0	...	0

Cummins

0.95
0
1
0
0

0.96
0
1
0
0

Dhoni

Australia

0.02
0.97
0
0.1
0

0.04
1
0
0
0

Zimbabwe

powerful concept

CODE BASICS

Issue with word2vec

Fixed embeddings

He didn't receive fair treatment

Fun fair in new York city this summer

fair → $\begin{bmatrix} 1 \\ 0.9 \\ 0.2 \\ 1 \\ 0.7 \end{bmatrix}$

BERT can generate contextualized embeddings

He didn't receive fair treatment → fair → $\begin{bmatrix} 1 \\ 0.9 \\ 0.2 \\ 1 \\ 0.7 \end{bmatrix}$

Tom deserves unbiased judgement → unbiased → $\begin{bmatrix} 1 \\ 0.8 \\ 0.2 \\ 1 \\ 0.6 \end{bmatrix}$

Fun fair in new York city this summer → fair → $\begin{bmatrix} 0 \\ 0.1 \\ 0 \\ 0.5 \\ 0.4 \end{bmatrix}$

Carnival was packed with fun activities → carnival → $\begin{bmatrix} 0 \\ 0.1 \\ 0.1 \\ 0.5 \\ 0.3 \end{bmatrix}$

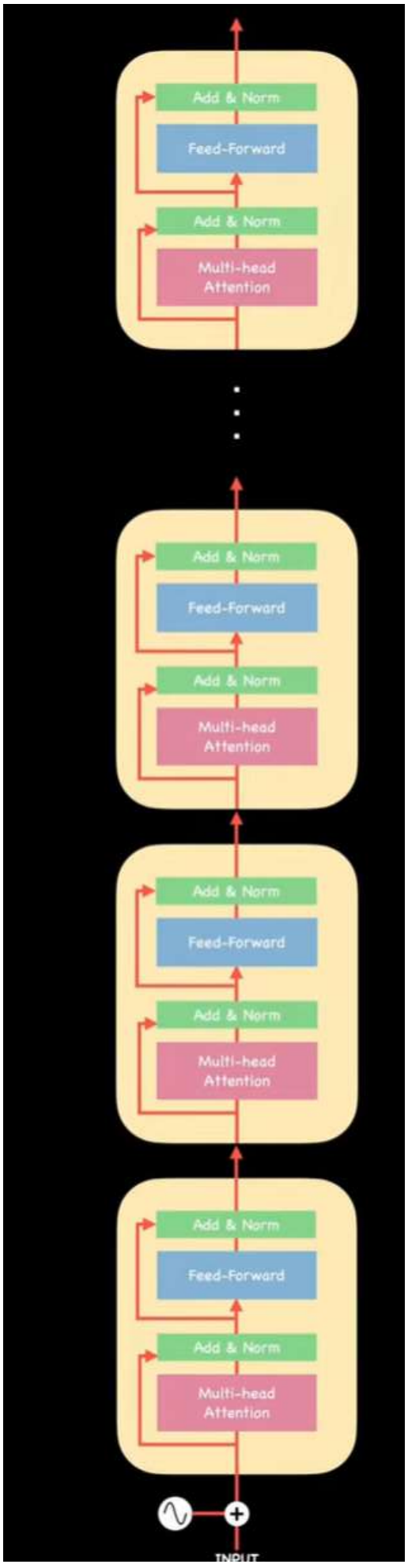
sentences and it will generate similar

BERT can generate embeddings for entire sentence

Amazing movie, I couldn't blink an eye for initial 45 minutes. It was that intense and interesting at the same time

$$\begin{bmatrix} 1 \\ 0.9 \\ 0.2 \\ 1 \\ 0.7 \\ \\ \dots \\ 0.1 \end{bmatrix}$$

768 length


Add & Norm
Feed-Forward
Add & Norm
Multi-head Attention
Add & Norm
Feed-Forward
Add & Norm
Multi-head Attention
Add & Norm
Feed-Forward
Add & Norm
Multi-head Attention
Add & Norm
Feed-Forward
Add & Norm
Multi-head Attention
INPUT

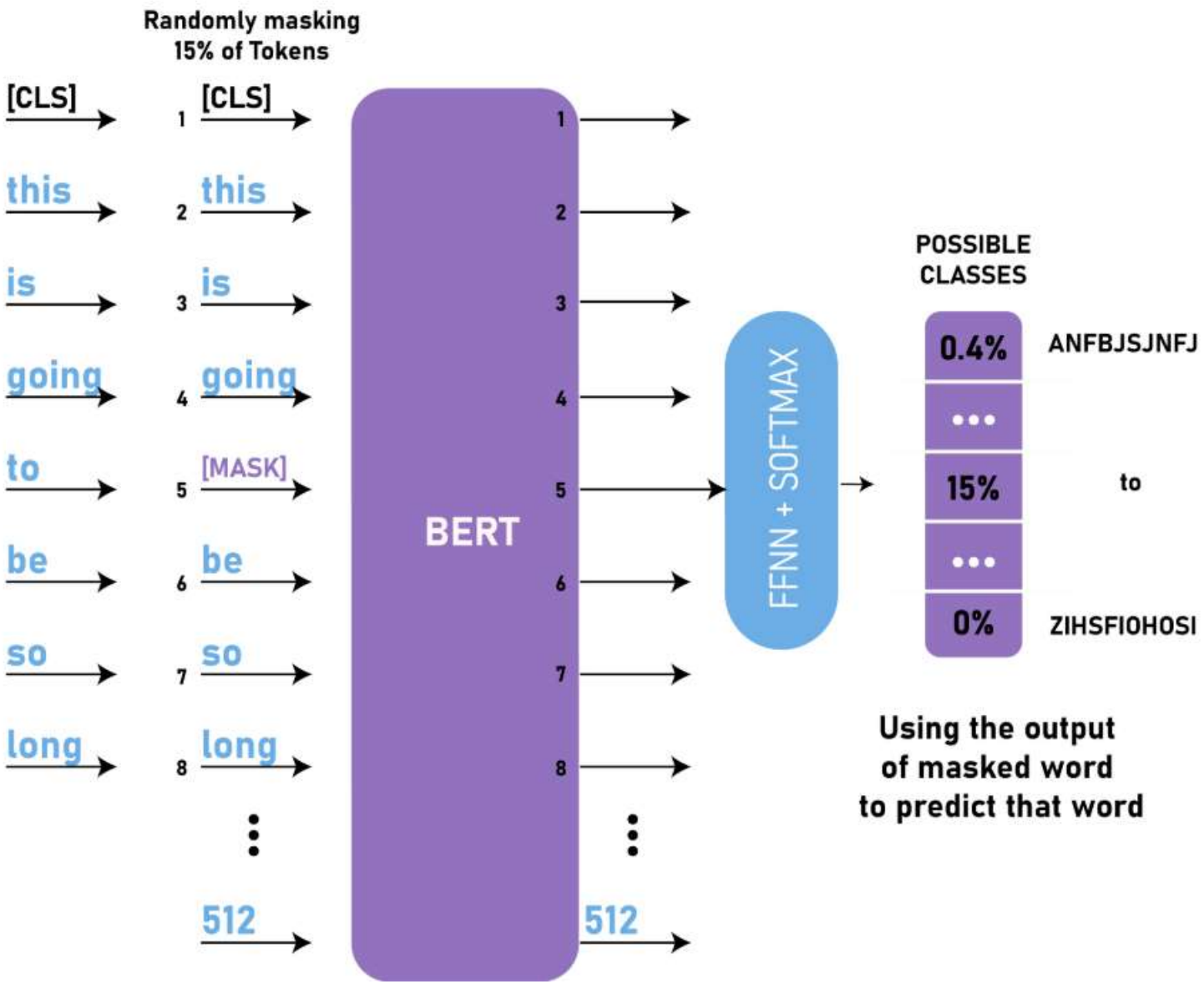
Traditional language models process text sequentially, either from left to right or right to left.

This method limits the model's awareness to the immediate context preceding the target word.

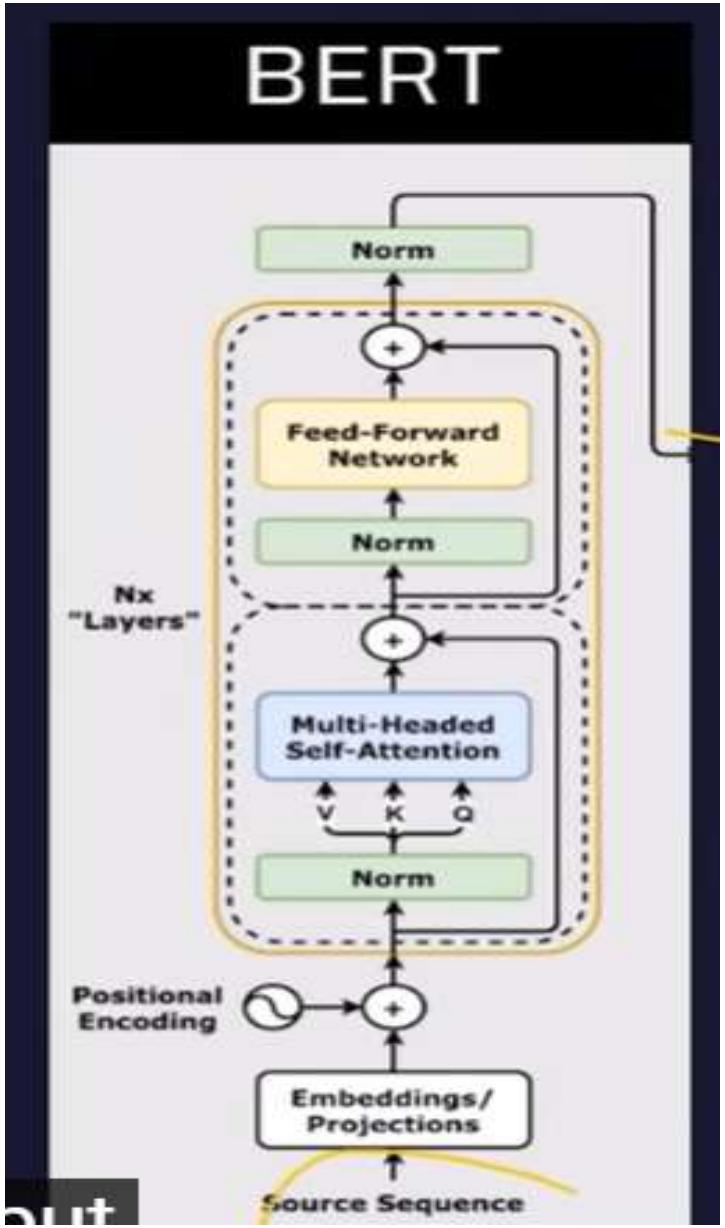
BERT uses a bi-directional approach considering both the left and right context of words in a sentence, instead of analyzing the text sequentially, BERT looks at all the words in a sentence simultaneously.

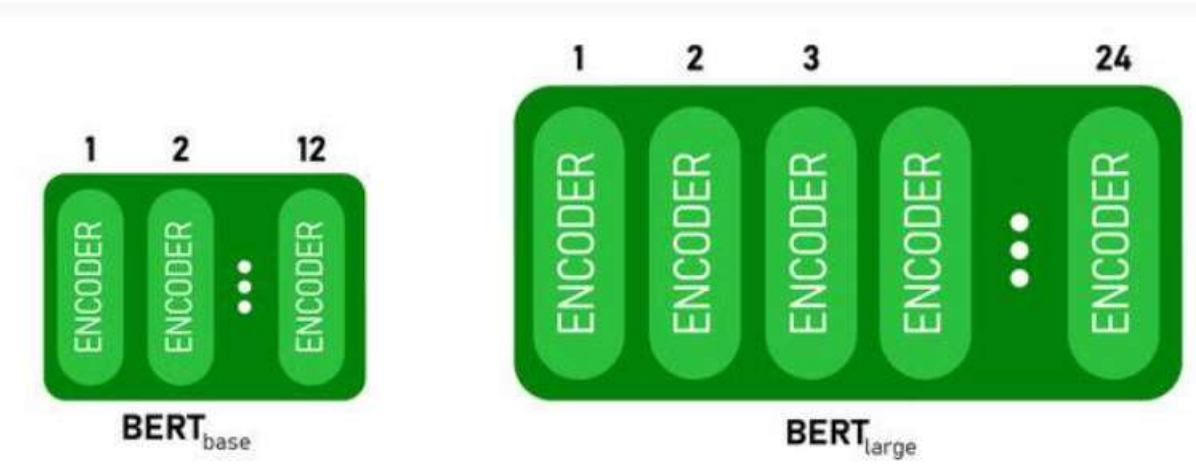
BERT Architecture

The architecture of BERT is a multilayer bidirectional transformer encoder which is quite similar to the transformer model. A transformer architecture is an encoder-decoder network that uses self-attention on the encoder side and attention on the decoder side.


Randomly masking
15% of Tokens
[CLS]
this
is
going
to
be
so
long
1
2
3
4
5
6
7
8
512
[CLS]
this
is
going
[MASK]
be
so
long
BERT
FFNN + SOFTMAX
POSSIBLE
CLASSES
0.4%
ANFBJSJNFJ
15%
to
0%
ZIHSFIOHOSI
Using the output
of masked word
to predict that word

1. $BERT_{BASE}$ has 12 layers in the Encoder stack while $BERT_{LARGE}$ has 24 layers in the Encoder stack. These are more than the Transformer architecture described in the original paper (6 encoder layers).
2. BERT architectures (BASE and LARGE) also have larger feedforward networks (768 and 1024 hidden units respectively), and more attention heads (12 and 16 respectively) than the Transformer architecture suggested in the original paper. It contains 512 hidden units and 8 attention heads.
3. $BERT_{BASE}$ contains 110M parameters while $BERT_{LARGE}$ has 340M parameters.


BERT
Norm
Feed-Forward Network
Norm
Nx "Layers"
Multi-Headed Self-Attention
V
K
Q
Norm
Positional Encoding
Embeddings/ Projections
Source Sequence

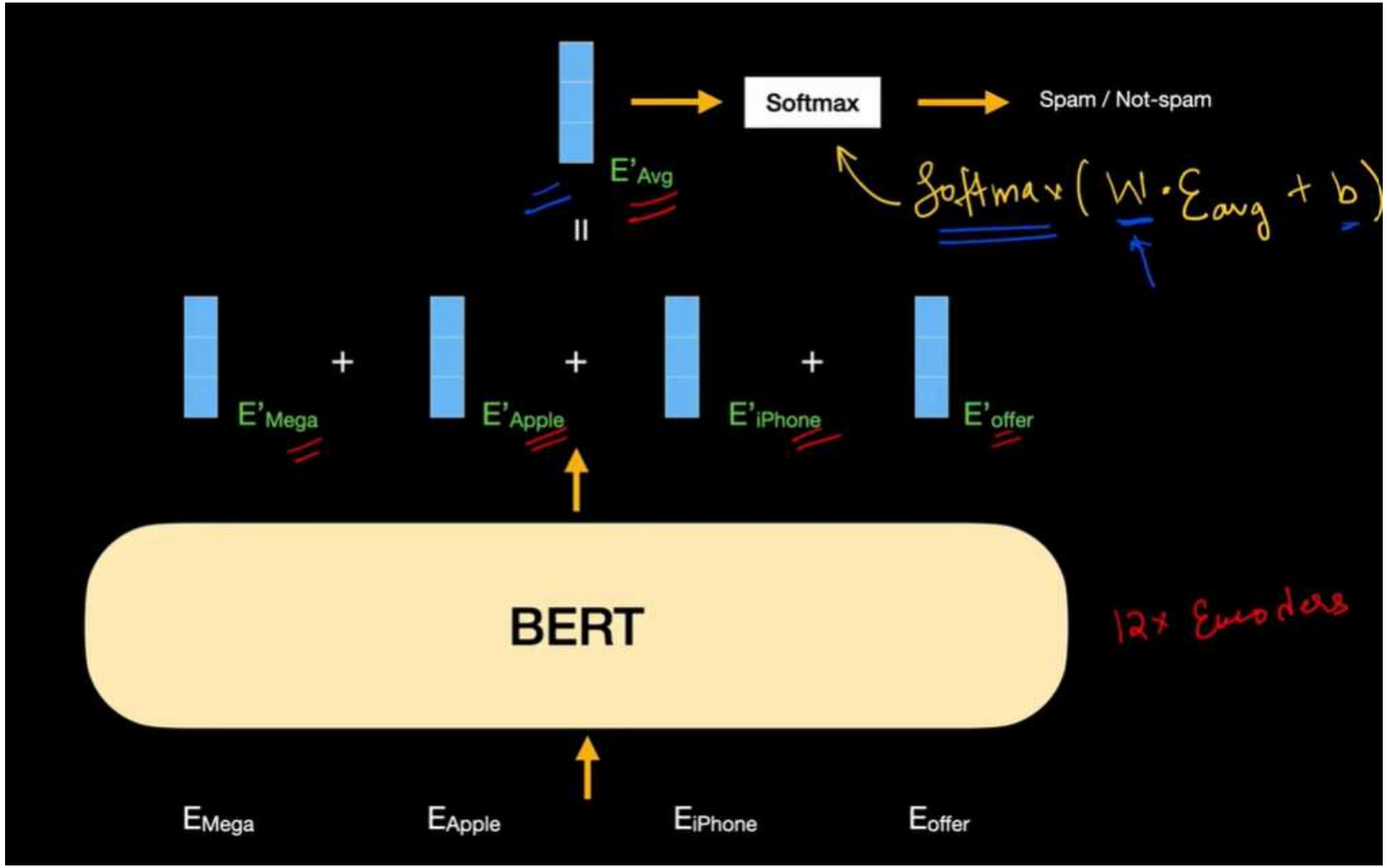


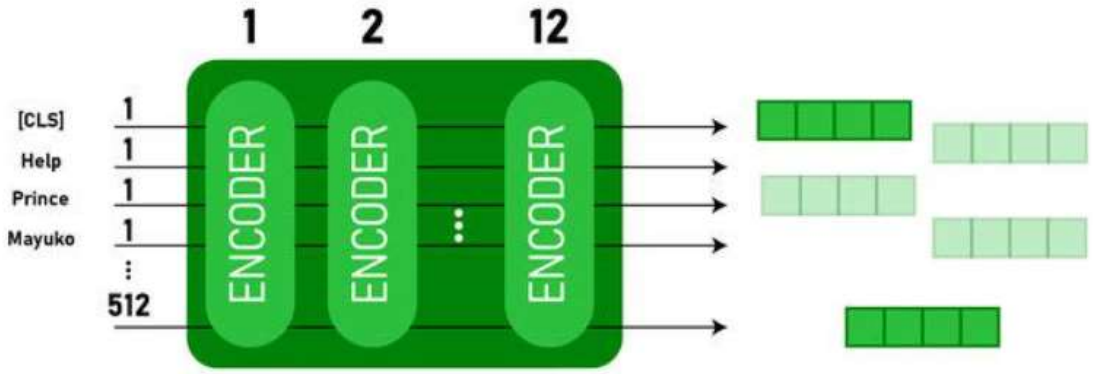
BERT BASE and BERT LARGE architecture

This model takes the CLS token as input first, then it is followed by a sequence of words as input. Here CLS is a classification token.

It then passes the input to the above layers. Each layer applies self-attention and passes the result through a feedforward network after then it hands off to the next encoder. The model outputs a vector of hidden size (768 for BERT BASE).

If we want to output a classifier from this model we can take the output corresponding to the CLS token.


Softmax
Spam / Not-spam
E'Avg
=
E'Mega
+
E'Apple
+
E'iPhone
+
E'offer
BERT
EMega
EApple
EiPhone
Eoffer



BERT output as Embeddings

How to use BERT model in NLP?

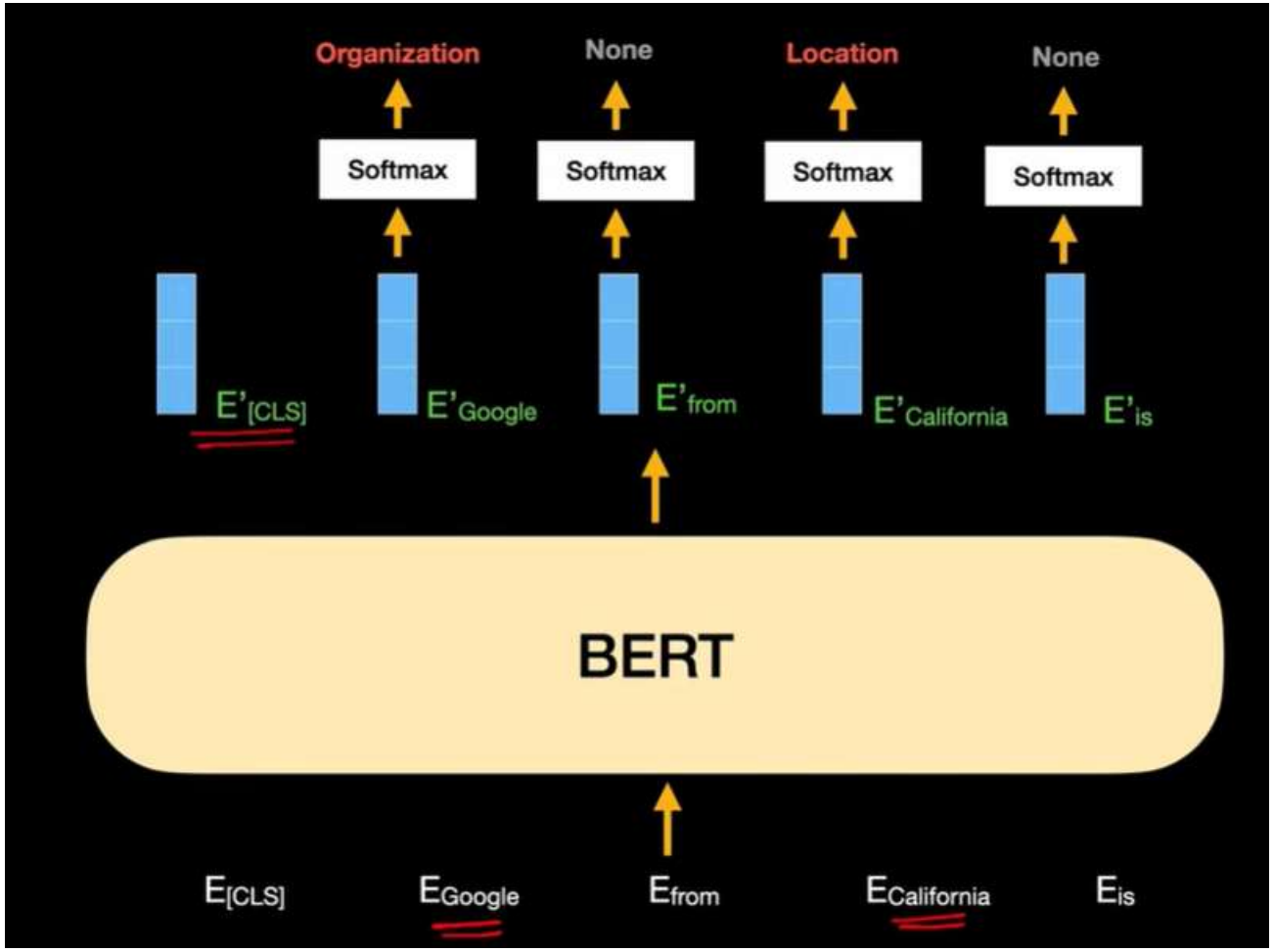
BERT can be used for various natural language processing (NLP) tasks such as:

Sentence-level classification

- Spam detection
- Sentiment analysis
- Movie rating prediction

Token-level classification

- Named Entity Recognition (NER)
- Part-of-Speech (POS) tagging


Organization
None
Location
None
Softmax
Softmax
Softmax
Softmax
E'[CLS]
E'Google
E'from
E'California
E'is
BERT
E[CLS]
EGoogle
Efrom
ECalifornia
Eis

1. Classification Task

- BERT can be used for classification task like sentiment analysis, the goal is to classify the text into different categories (positive/ negative/ neutral), BERT can be employed by adding a classification layer on the top of the Transformer output for the [CLS] token.
- The [CLS] token represents the aggregated information from the entire input sequence. This pooled representation can then be used as input for a classification layer to make predictions for the specific task.

2. Question Answering

- In question answering tasks, where the model is required to locate and mark the answer within a given text sequence, BERT can be trained for this purpose.
- BERT is trained for question answering by learning two additional vectors that mark the beginning and end of the answer. During training, the model is provided with questions and corresponding passages, and it learns to predict the start and end positions of the answer within the passage.

3. Named Entity Recognition (NER)

- BERT can be utilized for NER, where the goal is to identify and classify entities (e.g., Person, Organization, Date) in a text sequence.
- A BERT-based NER model is trained by taking the output vector of each token form the Transformer and feeding it into a classification layer. The layer predicts the named entity label for each token, indicating the type of entity it represents.

Generative Pre-trained Transformer (GPT)

Generative Pre-trained Transformer (GPT) is a large language model that can understand and produce human-like text.

It works by learning patterns, meanings and relationships between words from massive amounts of data.

Once trained, GPT can perform various language-related tasks such as writing, summarizing, answering questions and even coding all from a single model.

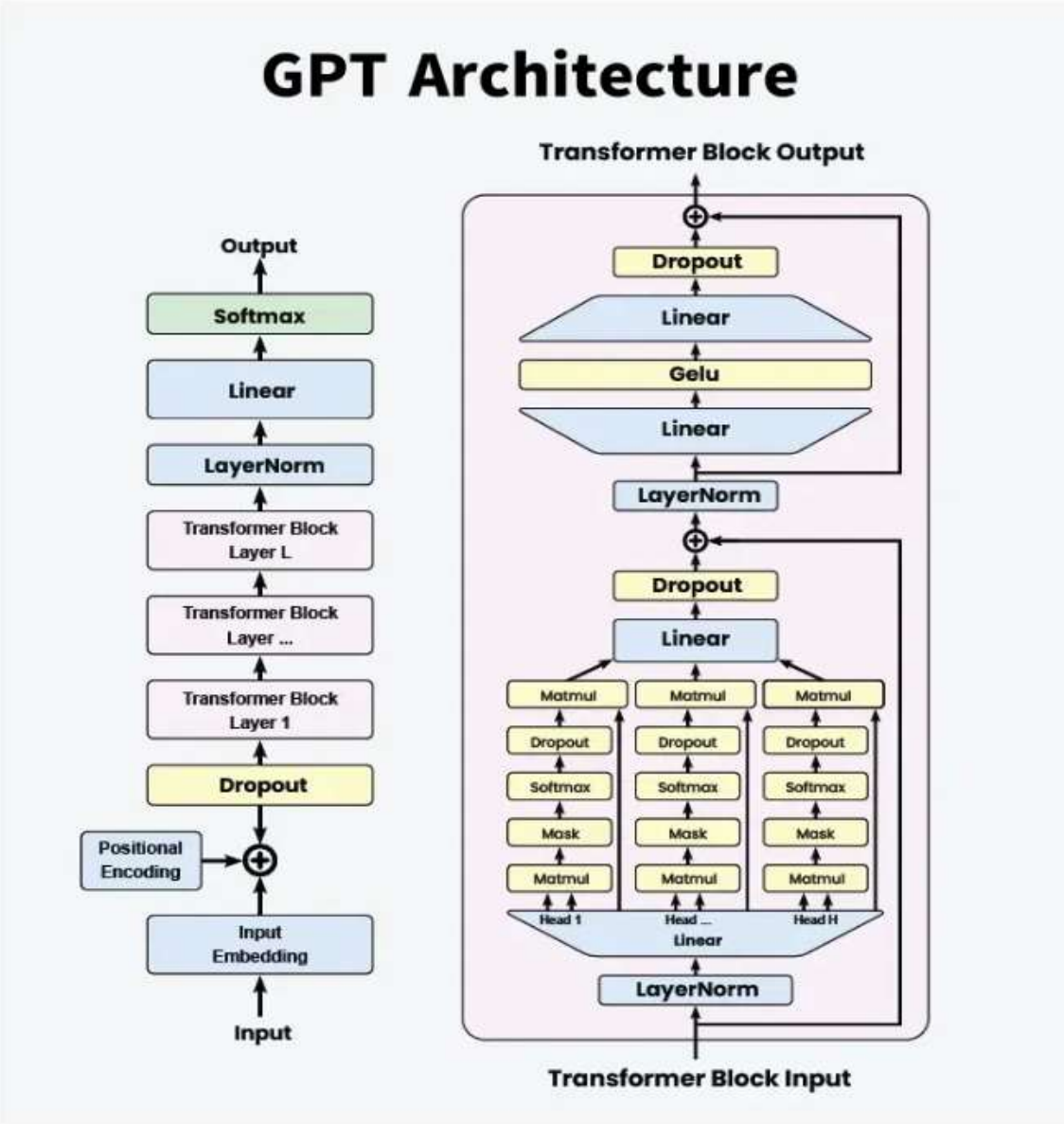
How GPT Works

GPT models are built upon the transformer architecture, introduced in 2017, which uses self-attention mechanisms to process input data in parallel, allowing for efficient handling of long-range dependencies in text. The core process involves:

2. Pre-training: The model is trained on vast amounts of text data to learn language patterns, grammar, facts and some reasoning abilities.
3. Fine-tuning: The pre-trained model is further trained on specific datasets with human feedback to align its responses with desired outputs.

This two-step approach enables GPTs to generate coherent and contextually relevant responses across a wide array of topics and tasks.

Architecture



1. Input Embedding

- Input: The raw text input is tokenized into individual tokens (words or subwords).
- Embedding: Each token is converted into a dense vector representation using an embedding layer.

2. Positional Encoding: Since transformers do not inherently understand the order of tokens, positional encodings are added to the input embeddings to retain the sequence information.

3. Dropout Layer: A dropout layer is applied to the embeddings to prevent overfitting during training.

4. Transformer Blocks

- LayerNorm: Each transformer block starts with a layer normalization.
- Multi-Head Self-Attention: Multi-Head Self-Attention are core component where the input passes through multiple attention heads.
- Add & Norm: The output of the attention mechanism is added back to the input (residual connection) and normalized again.
- Feed-Forward Network: A position-wise Feed-Forward Network is applied, typically consisting of two linear transformations with a GeLU activation in between.
- Dropout: Dropout is applied to the feed-forward network output.

5. Layer Stack: The transformer blocks are stacked to form a deeper model, allowing the network to capture more complex patterns and dependencies in the input.

6. Final Layers

- LayerNorm: LayerNorm is final layer normalization is applied.
- Linear: The output is passed through a linear layer to map it to the vocabulary size.
- Softmax: A Softmax layer is applied to produce the final probabilities for each token in the vocabulary.

BERT’s variants and adaptations

But this is only a part of the story. The success of BERT is greatly due to its open-source nature, which has allowed developers to access the source code of the original BERT and create new features and improvements.

This has resulted in a good number of BERT’s variants. Below, you can find some of the most well-known variants:

RoBERTa. Short for “Robustly Optimized BERT Approach", RoBERTa is a BERT variant created by Meta in collaboration with Washington University. Considered a more powerful version than the original BERT, RoBERTa was trained with a dataset 10 times bigger than the one used to train BERT.

- **DistilBERT.** Since the launch of the first LLMs in the late 2010s, there has been a consolidated trend of building bigger and heavier LLMs. This makes sense, as there seems to be a direct relationship between model size and model accuracy.
- Yet, it’s also true that the bigger the model, the more resources it requires to run, and hence, fewer people can afford to use it. DistilBERT aims at making BERT more accessible by offering a smaller, faster, cheaper, and lighter variant.
- Based on the architecture of the original BERT, DistilBERT uses knowledge distillation techniques during pre-training to reduce the size by 40% while retaining 97% of its language understanding capabilities and being 60% faster.

- **ALBERT.** Stands for A Lite BERT, ALBERT was specifically designed to increase the efficiency of BERTlarge during pre-training.
- As training bigger models often results in memory limitations, longer training times, and unexpected model degradation, ALBERT creators developed two parameter-reduction techniques to reduce memory consulting and increase speed during training.